FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Felipe Yuzo Tanaka Shida

Produto: **AniTrack**
https://github.com/FeYuzo/Animes-API.git

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 6
3.2 Onde roda 8
3.3 Fluxo de uma requisição 9
3.4 Fluxo de assinatura 13
3.5 Modelo de dados do domínio 15
3.6 Segurança 15
3.7 Observabilidade, CI/CD e documentação da API 16
3.8 Camadas IaaS, PaaS e SaaS do projeto 16
4 Quadro de entregas 16
5 Atividades realizadas 17
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 17
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 18
5.3 Atividade 9 — Fork e plano de negócio 20
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 23
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 27
6 Considerações finais 31
7 Declaração de autoria 31

1 Introdução

O presente trabalho foi desenvolvido no âmbito da disciplina ISW055 — Introdução à Computação em Nuvem,ministrada pelo Prof. Allan Siriani. A disciplina tem como propósito capacitar os estudantes nos fundamentos arquiteturais, operacionais e de governança de aplicações modernas nativas em nuvem (cloud-native), com ênfase no desacoplamento por microsserviços, conteinerização com Docker, observabilidade, mensageria orientada a eventos, armazenamento de objetos e integração de serviços gerenciados em nuvem. Como ponto de partida prático, as primeiras etapas do curso utilizaram um projeto-base de catálogo de filmes para consolidar conceitos fundamentais de APIs REST e persistência. Essa base foi progressivamente reformulada, expandida e refatorada até evoluir para o AniTrack, uma plataforma completa de Software como Serviço (SaaS) projetada com arquitetura de microsserviços e modelo de negócio real.

O AniTrack foi idealizado para solucionar a dispersão de informações e a desorganização enfrentadas por espectadores e fãs de animes (especialmente entre 16 e 35 anos) que acompanham simultaneamente múltiplos lançamentos sazonais da televisão japonesa (seasonal simulcasts). Atualmente, esse público recorre a soluções improvisadas, como blocos de notas, mensagens soltas no WhatsApp e planilhas manuais ou a plataformas legadas (como o MyAnimeList) que sofrem com interfaces lentas, poluição de banners de anúncios e pouca automação. A consequência direta é o esquecimento do progresso dos episódios, perda de datas de estreia e frequente exposição a spoilers em redes sociais. O AniTrack elimina esses atritos ao centralizar, em uma interface veloz e limpa, o histórico de animes já assistidos, a fila de obras que pretende ver (plan to watch), a vitrine de títulos favoritos e uma grade semanal de lançamentos sincronizada em tempo real com a API da AniList.

Para viabilizar a sustentabilidade do produto, a plataforma adota o modelo de negócios Freemium, integrando um fluxo de assinaturas recorrentes com o Stripe Billing. Usuários da camada gratuita contam com rastreamento irrestrito de seus animes, enquanto os assinantes do plano pago (Otaku Pro) contam com recursos exclusivos de prestígio e personalização comunitária. Como upload de fotos e banners animados (GIFs) hospedados em Object Storage (MinIO/S3), destaque de animes no perfil, contadores de tempo assistido e backup completo da biblioteca em formato JSON. Essa lógica comercial é suportada por uma infraestrutura dividida em microsserviços independentes (Catálogo/Backend, Serviço de Autenticação e Serviço de Auditoria), que garantem segurança e isolamento de responsabilidades.

Este relatório documenta detalhadamente todo o projeto de engenharia de software e computação em nuvem que deu vida ao AniTrack. O documento está estruturado em tópicos lógicos que cobrem: (1) a modelagem do negócio, personas, concorrentes e proposta de valor; (2) o detalhamento das rotas e contratos da API; (3) a topologia de microsserviços, isolamento em redes internas Docker e matriz de comunicação inter-serviços; (4) o modelo de dados relacional (MariaDB) e scripts versionados de migração; (5) as políticas de autenticação JWT e autorização baseada em papéis (RBAC com os níveis usuario, critico e admin); (6) a trilha imutável de eventos de auditoria com Redis Streams; (7) o armazenamento de mídia no MinIO com validação de segurança; e, por fim, (8) as evidências de testes práticos, rotinas de deploy e instruções para reprodução do ambiente.

2 Metodologia

O desenvolvimento das atividades práticas e a evolução arquitetural do projeto foram conduzidos utilizando um ecossistema moderno voltado à computação em nuvem e microsserviços. No ambiente

de desenvolvimento local, foram empregados o Visual Studio Code, Node.js (Express), Git e o Docker Desktop com Docker Compose para orquestração e execução dos containers independentes (`catalog`, `auth-service`, `log-service`, `mariadb`, `redis` e `minio`). Para validação de serviços externos em nuvem, utilizou-se o Stripe em modo de testes (com CLI local para recepção de webhooks) e o Mailtrap como servidor SMTP para envio transaciona l de e-mails com links temporários de recuperação de senha. No ambiente de produção e entrega, a aplicação foi implantada sobre a infraestrutura da disciplina em um servidor Linux gerenciado via Portainer, operando sob uma rede interna Docker isolada (`internal-net`) e exposta publicamente através do subdomínio individual atribuído ao aluno (`http://felipe-shida.siriani.com.br`).

As atividades 7 e 8 foram elaboradas originalmente sobre o repositório público do catálogo de filmes utilizado no 1º bimestre. Na Atividade 9, visando estruturar a nova identidade do produto como SaaS independente sem abrir mão da rastreabilidade acadêmica, realizou-se a clonagem espelhada de todo o histórico (`git clone --bare` seguido de `git push --mirror`) para um novo repositório oficial no GitHub (`FeYuzo/Animes-API`). A partir desse ponto, durante as atividades 10 e 11, o sistema foi integralmente remodelado e desacoplado para dar lugar ao AniTrack: integraram-se os novos fluxos de animes em simulcast via AniList GraphQL, o controle de permissões centralizado (RBAC), a esteira assíncrona de auditoria com Redis Streams, o armazenamento de fotos e banners validados por *magic bytes* no MinIO Object Storage (S3) e a monetização via Stripe Billing, mantendo a mesma stack de containers no Portainer e o mesmo subdomínio de homologação.

A aferição das datas e horários oficiais de cada entrega seguiu critérios rigorosos de auditoria de versionamento. Os registros cronológicos foram extraídos diretamente do histórico de commits do repositório local por meio do comando `git log` (com formatação ISO), conferidos e auditados na interface web do GitHub através dos identificadores de commit (hashes SHA) e, a partir da Atividade 9, vinculados às respectivas *issues* e mensagens de entrega contendo menção direta ao professor orientador (`@siriani`).

As evidências técnicas e operacionais apresentadas neste relatório foram coletadas por meio de capturas de tela padronizadas e registros de logs em tempo de execução. As evidências englobam: **(a)** a documentação do `README.md` contendo os dados do projeto e a menção ao professor; **(b)** o histórico de commits do Git atestando a linearidade do desenvolvimento; **(c)** a execução da Single Page Application (SPA) e dos endpoints REST no navegador tanto em desenvolvimento local quanto em produção; e **(d)** a validação dos painéis de infraestrutura, incluindo a listagem de stacks ativas no Portainer, as cobranças simuladas no painel da Stripe, os e-mails transacionais recebidos no Mailtrap e o fluxo imutável de eventos registrados no Redis Streams.

As atividades 7 e 8 foram feitas no repositório público do catálogo de filmes, o mesmo do 1º bimestre. Na atividade 9, esse repositório foi copiado com todo o histórico (`git clone --bare` + `git push --mirror`) para um repositório novo, com o nome do produto, onde o SaaS foi desenvolvido nas atividades 10 e 11, na mesma stack do Portainer e no mesmo domínio. A data e a hora de cada entrega foram extraídas do histórico de commits (`git log`), conferidas na página do commit no GitHub e, da atividade 9 em diante, na issue de entrega que menciona o professor. Os prints mostram o README com a menção ao professor, o histórico de commits e o sistema em execução.

3 Arquitetura do SaaS

A arquitetura do AniTrack foi concebida e implantada seguindo o modelo de microsserviços desacoplados, conteinerizados via Docker e orquestrados pela stack no Portainer. O sistema em produção reflete

estritamente a topologia física da infraestrutura, onde apenas o serviço de entrada possui exposição de portas ao tráfego externo, mantendo todos os demais componentes isolados em uma rede *bridge* interna (`internal-net`).

3.0.1 Componentes da Stack em Execução

1. **Camada de Ingress e Aplicação Principal (`catalog`):** Container Node.js/Express operando internamente na porta `:3000` e mapeado para a porta `:8206` no host (`https://felipe-shida-isw055.lapps.studio`). É o único ponto de contato com a internet. Ele serve a Single Page Application (HTML/CSS/JS) para o cliente, atende aos endpoints da API REST de gerenciamento de animes (Assistidos, Fila e Favoritos), executa streaming de avatares armazenados no MinIO e consome a API GraphQL pública da AniList para catálogo e simulcast.
2. **Microsserviço de Identidade e Autenticação (`auth-service`):** Container Node.js operando na porta interna `:4000` (sem portas mapeadas para o host). Responsável pelo ciclo de vida dos usuários: registro, login, emissão/verificação de tokens JWT criptografados, enforcement centralizado de permissões RBAC (`usuario`, `critico`, `admin`) e disparo transacional de e-mails de recuperação de senha via SMTP.
3. **Microsserviço de Logs e Auditoria (`log-service`):** Container Node.js operando na porta interna `:5000` (sem portas mapeadas para o host). Recebe eventos de comportamento e trilhas de segurança em chamadas internas e os insere com alta taxa de transferência no Redis.
4. **Armazenamento em Memória e Mensageria (`redis`):** Container oficial `redis:7-alpine` executando na porta interna `:6379`. Opera com persistência ativa em disco via *Append-Only File* (`--appendonly yes`) através do volume nomeado `redis-data`, persistindo o stream imutável `audit:events`.
5. **Armazenamento de Objetos S3 (`minio`):** Container MinIO Object Storage executando com portas internas `:9000` (API S3) e `:9001` (Console Web), persistido no volume Docker `minio-data`. Gerencia o bucket `perfil-usuarios` para armazenar fotos de perfil e banners de assinantes, validados por *magic bytes*.
6. **Persistência Relacional Gerenciada (MariaDB):** Banco de dados MariaDB hospedado no host da máquina virtual (`35.226.64.52:3306`), acessado exclusivamente pelos microsserviços através da base `IAC_2026_02_felipe_shida` para consistência transacional de usuários, listas e comentários.
7. **Serviços Externos Integrados:**
   - **Stripe Billing:** Plataforma de pagamento em nuvem consumida via HTTPS pelo `catalog` para processamento do checkout do plano Otaku Pro e recebimento de webhooks assinados.
   - **AniList GraphQL API:** Fonte externa (`https://graphql.anilist.co`) consumida para busca de metadados deanimes e datas de simulcast.
   - **Mailtrap SMTP:** Servidor de mensageria TLS (`sandbox.smtp.mailtrap.io:2525`) consumido pelo `auth-service` para envio de tokens temporários de redefinição de senha.

3.0.2 Matriz de Comunicação entre Serviços

Origem	Destino	Protocolo / Porta	Finalidade da Comunicação
Navegador (Cliente)	`catalog`	HTTPS / `:443 -> :8206`	Carregamento da SPA, chamadas REST com header `Authorization: Bearer <JWT>`

Origem	Destino	Protocolo / Porta	Finalidade da Comunicação
`catalog`	`auth-service`	HTTP / `:4000` (Interno)	Validação de tokens e checagem de permissões RBAC (`/api/auth/authorize`)
`catalog` / `auth`	`log-service`	HTTP / `:5000` (Interno)	Envio assíncrono de eventos de auditoria e segurança (`POST /api/logs/events`)
`log-service`	`redis`	RESP / `:6379` (Interno)	Gravação imutável no stream `audit:events` via comando `XADD`
`catalog` / `auth`	MariaDB	MySQL / `:3306` (VM Host)	Persistência de usuários, listas de animes e comentários
`catalog`	`minio`	S3 API / `:9000` (Interno)	Upload de imagens multipart e streaming seguro de fotos e banners
`catalog`	AniList GraphQL	HTTPS / `:443` (Externo)	Consulta da grade de animes da temporada e simulcasts
`catalog`	Stripe API	HTTPS / `:443` (Externo)	Criação de sessões de checkout e recepção de webhooks de assinatura
`auth-service`	Mailtrap SMTP	SMTP / `:2525` (Externo)	Disparo de e-mails transacionais com links de redefinição de senha

3.1 Visão geral e componentes

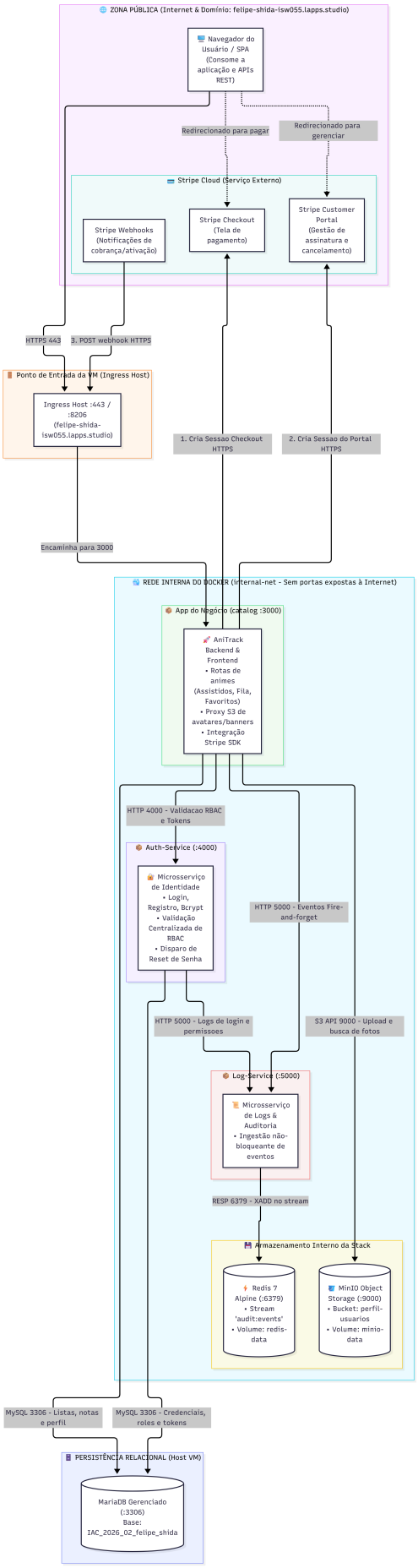


Figura 1: Diagrama de componentes e microsserviços do SaaS

Componente	Responsabilidade	Tecnologia / imagem	Público?
app	telas, API do negócio, landing page	Node.js / Express (`node:20-alpine`)	sim (domínio)
auth-service	login, cadastro, papéis (RBAC)	Node.js / Express (`node:20-alpine`)	não (rede interna)
log-service	eventos de negócio e auditoria	Node.js / Express (`node:20-alpine`)	não (rede interna)
Redis	armazenamento dos logs	Redis (`redis:7-alpine`)	não (rede interna)
MariaDB	dados do domínio, usuários, planos	MariaDB (`mariadb:10.11`)	não (rede interna)
MinIO	arquivos do domínio	MinIO S3 (`minio:latest`)	não (rede interna)
Stripe (externo)	checkout, assinaturas, portal, webhooks	serviço SaaS	webhook → app

3.2 Onde roda

A solução permanece hospedada na mesma infraestrutura de nuvem e na mesma máquina virtual utilizada desde o início do semestre letivo. O ciclo de deploy, orquestração e gerenciamento do ciclo de vida dos containers é realizado através do **Portainer CE**, garantindo continuidade operacional, isolamento em rede bridge interna e facilidade de manutenção.

Dados de Implantação e Infraestrutura:

- **Máquina Virtual / Host:** Servidor Linux gerenciado na nuvem da disciplina (`35.226.64.52`).
- **Nome da Stack no Portainer:** `felipe-shida` (ou `anitrack-stack`).
- **Repositório de Origem (Build Contínuo):** Repositório oficial no GitHub

(`https://github.com/FeYuzo/Animes-API`), apontando para a branch `main`.

- **URL Pública Oficial do Sistema:** https://felipe-shida-isw055.lapps.studio/index.html
- **Painel de Gestão (Portainer):** http://35.226.64.52:9000

3.2.1 Containers da Stack e Alocação de Recursos

A aplicação opera dividida em microsserviços desacoplados e containers de suporte conectados à rede interna isolada `internal-net`. Apenas o container de catálogo/frontend expõe porta ao tráfego do ingress público:

Container	Responsabilidade no SaaS	Exposição de Portas	Limite de Memória
`catalog`	Backend principal, API REST de animes e SPA do Frontend	`8206:3000` (Ingress Público)	512 MB

Container	Responsabilidade no SaaS	Exposição de Portas	Limite de Memória
`auth-service`	Microsserviço de autenticação, JWT, RBAC e envio de e-mails	`4000` (Rede interna)	256 MB
`log-service`	Microsserviço de ingestão de logs e eventos comportamentais	`5000` (Rede interna)	128 MB
`redis`	Banco em memória Redis 7 Alpine com persistência AOF	`6379` (Rede interna)	128 MB
`minio`	Object Storage compatível com AWS S3 para avatares e banners	`9000 / 9001` (Interna)	256 MB

Nota: O banco de dados relacional MariaDB é compartilhado no nível do host pela infraestrutura da disciplina (`35.226.64.52:3306`), acessado pelos serviços por credenciais de conexão isoladas.

3.2.2 Nomes das Variáveis de Ambiente Configuradas

Em estrito alinhamento às boas práticas de segurança em nuvem (Twelve-Factor App), nenhuma credencial ou segredo sensível é versionado no repositório. As configurações são injetadas dinamicamente na stack do Portainer através das seguintes variáveis de ambiente:

- **Roteamento e Rede:** `PORT`, `APP_URL`, `AUTH_SERVICE_URL` e `LOG_SERVICE_URL`.
- **Banco de Dados Relacional (MariaDB):** `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD` e `DB_NAME`.
- **Segurança e Sessão:** `JWT_SECRET`.
- **Mensageria e Auditoria (Redis):** `REDIS_HOST`, `REDIS_PORT` e `STREAM_KEY`.
- **Armazenamento de Objetos (MinIO S3):** `MINIO_ENDPOINT`, `MINIO_PORT`, `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`,

`MINIO_BUCKET`, `MINIO_PUBLIC_URL`, `MINIO_ROOT_USER` e `MINIO_ROOT_PASSWORD`.

- **Serviço Transacional de E-mail (SMTP / Mailtrap):** `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS` e

`SMTP_FROM`.

- **Motor de Pagamento e Assinaturas (Stripe):** `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID` e `STRIPE_WEBHOOK_SECRET`.
- **Integração Externa com API GraphQL (AniList):** `ANILIST_CLIENT_ID` e `ANILIST_CLIENT_SECRET`.

3.3 Fluxo de uma requisição

Para demonstrar a integração de ponta a ponta entre a camada cliente, os microsserviços desacoplados e os sistemas de persistência, descreve-se a seguir o ciclo de vida completo de uma das ações mais frequentes do AniTrack: **um usuário autenticado adiciona um anime ao seu histórico pessoal de animes assistidos, registrando os episódios vistos e sua nota pessoal**.

3.3.1 Ação Escolhida: Adicionar Anime aos Assistidos (`POST /api/animes/assistidos`)

Passo 1: **Navegador (Cliente / SPA):** O usuário logado interage com o card de uma obra no catálogo (ex.: *Sousou no Frieren*) e clica em "Marcar como Assistido", definindo os episódios assistidos (28) e a nota pessoal (10). A Single Page Application (JavaScript) intercepta o evento, monta o payload JSON e despacha uma requisição assíncrona `POST /api/animes/assistidos` contendo o cabeçalho `Authorization: Bearer <token_jwt>`.

Passo 2: **Domínio Público & Ingress:** A requisição viaja criptografada sob HTTPS até o domínio público oficial (`https://felipe-shida-isw055.lapps.studio`). O proxy reverso/ingress da infraestrutura da disciplina recebe o tráfego na porta 443 e faz o encaminhamento de porta para a porta `:8206` no host da máquina virtual (`35.226.64.52`), entregando a requisição à porta interna `:3000` do container `catalog` na rede interna Docker (`internal-net`).

Passo 3: **Aplicação Backend (`catalog`):** O servidor Node.js/Express intercepta o tráfego. Antes de atingir o controlador de negócio, a rota passa pelo middleware de segurança `authenticate` (`auth.js`), que extrai o token JWT do cabeçalho e valida sua integridade criptográfica e validade temporal por meio do segredo compartilhado (`JWT_SECRET`).

Passo 4: **Autorização e Papéis (`auth-service`):** Para garantir a consistência das regras de acesso (RBAC), o backend consulta o microsserviço `auth-service:4000` via chamada HTTP interna (`POST /api/auth/authorize`), verificando se o usuário possui o papel `usuario` ou `otaku` com permissão para gerenciar suas próprias listas (`listas:gerenciar_proprio`). Confirmada a permissão, o objeto do usuário autenticado é injetado em `req.user`.

Passo 5: **Persistência Relacional (MariaDB):** O fluxo avança para o controlador de negócio (`animeController.addWatched`). O controlador sanitiza os parâmetros de entrada e executa uma query parametrizada no banco relacional MariaDB (`35.226.64.52:3306`) através do pool de conexões:

```
INSERT INTO animes_assistidos (usuario_id, anime_id, titulo, poster_url,
episodios_vistos, total_episodios,
nota,
status)
VALUES
(?, ?, ?, ?, ?, ?, ?, ?)
ON DUPLICATE KEY UPDATE episodios_vistos = VALUES(episodios_vistos), nota =
VALUES(nota);
```

O MariaDB efetiva a escrita respeitando as chaves estrangeiras e a restrição de unicidade (`usuario_id`, `anime_id`), retornando a confirmação com sucesso.

Passo 6: **Auditoria Comportamental Assíncrona (`log-service` & Redis):** Imediatamente após a gravação relacional, o backend aciona o serviço de auditoria (`auditService.logAuditEvent`) em background (**fire-and-forget**, não-bloqueante). Uma requisição interna é disparada para o container `log-service:5000` (`POST/api/logs/events`) com os detalhes do evento (`acao: "anime_marcado_assistido"`, IP do cliente e metadados). O `log-service` recebe o evento e o grava no stream `audit:events` do container `redis:6379` via comando `XADD`, garantindo rastreabilidade imutável sem onerar o tempo de resposta do usuário.

Passo 7: **Resposta HTTP e Atualização Visual:** O backend busca o registro consolidado e devolve ao navegador uma resposta `HTTP 201 Created` contendo os dados do anime salvo. A interface SPA processa o retorno, exibe uma notificação de sucesso e atualiza o card do anime dinamicamente no DOM com a badge de concluído e a nota atribuída, sem que a página precise ser recarregada.

3.3.2 Diagrama da Arquitetura em Produção

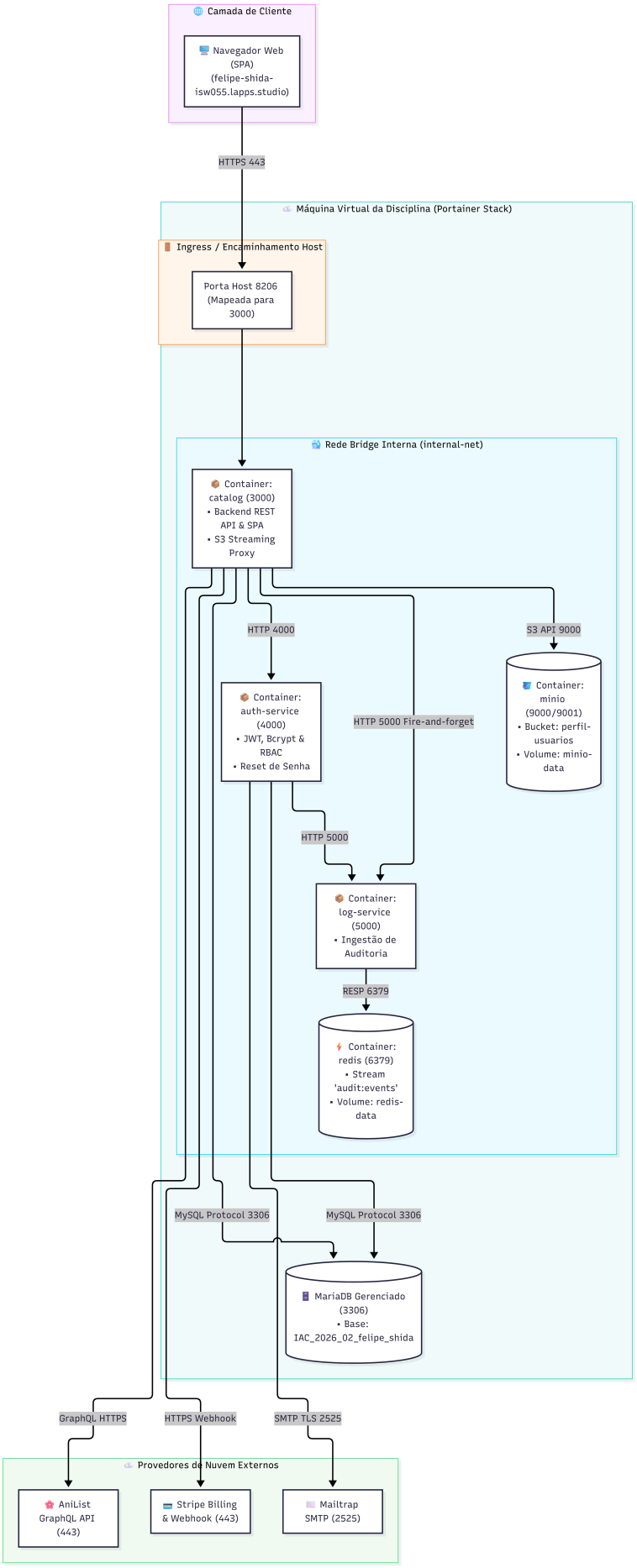


Figura 2: Topologia da arquitetura do AniTrack implantada na stack Docker

3.4 Fluxo de assinatura

3.4.1 1. Fluxo da Assinatura aos Limites

1. **Checkout (`mode=subscription`):** O usuário clica em "Assinar", a API gera uma sessão no Stripe com `client_reference_id` (ID do usuário) e

redireciona para a página hospedada de pagamento.

2. **Pagamento e Webhooks:** O Stripe processa o cartão e notifica a API de forma assíncrona via `POST /api/stripe/webhook` tratando os

principais eventos:

- `checkout.session.completed` e `invoice.payment_succeeded`: Pagamento confirmado; define `is_premium = TRUE` no MariaDB.
- `customer.subscription.deleted` e `invoice.payment_failed`: Cancelamento ou inadimplência; rebaixa para `is_premium = FALSE`.

3. **Limites aplicados na API:** Em cada requisição (`checkUserIsPremium`), a API valida o plano:
   - **Favoritos fixados:** Até 6 animes no Pro vs. 2 no Free (403 se exceder).
   - **Avatares e Banners:** GIFs animados e banners são exclusivos do Pro via validação de *magic bytes*.
   - **Exportação:** Download da biblioteca em JSON restrito ao plano Pro.

3.4.1.1 2. Validação da Assinatura e Idempotência

- **Validação Criptográfica:** O webhook é recebido com `express.raw()` para manter o payload binário exato. O método

`stripe.webhooks.constructEvent()` confere o cabeçalho `stripe-signature` com o segredo `STRIPE_WEBHOOK_SECRET` via HMAC-SHA256, descartando *payloads* adulterados ou requisições forjadas com `HTTP 400`.

- **Idempotência:** O Stripe adota entrega *"at least once"* e pode reenviar o mesmo evento. As atualizações no banco são baseadas em comandos

`UPDATE` por IDs únicos (`stripe_subscription_id` e `id`). Executar o mesmo evento múltiplas vezes mantém rigorosamente o mesmo estado, sem duplicar dados nem conceder vantagens indevidas.

3.4.1.2 3. Cancelamento pelo Portal do Cliente

O usuário clica em "Gerenciar Assinatura" e a API gera uma sessão do Stripe Customer Portal (`billingPortal.sessions.create`). No portal do Stripe, ele pode cancelar a assinatura diretamente. O Stripe dispara o evento `customer.subscription.deleted`, a API atualiza o MariaDB (`is_premium = FALSE`) e os limites do plano Free voltam a valer imediatamente na próxima requisição.

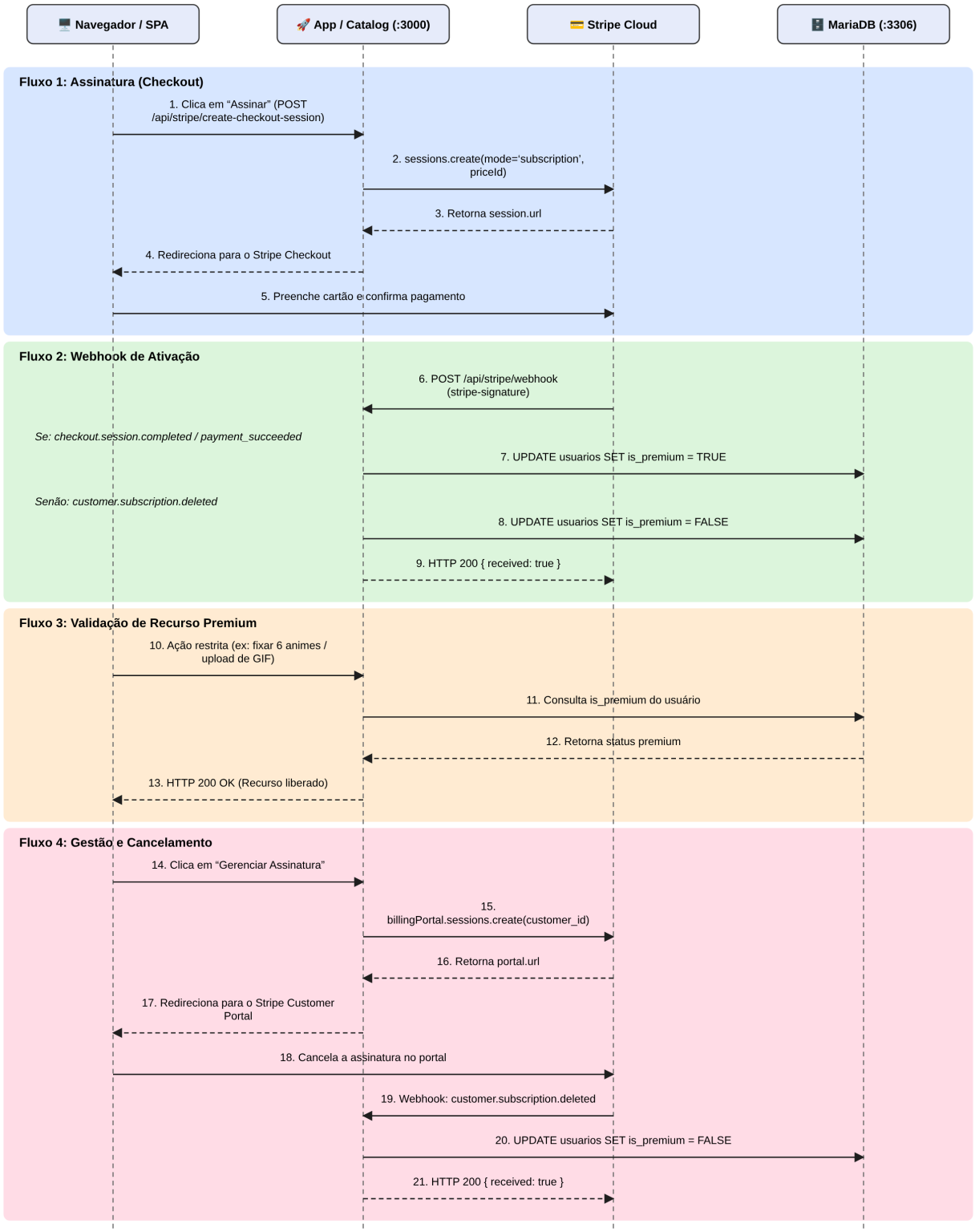


Figura 3: Fluxo de assinatura: checkout → webhook → plano no banco

3.5 Modelo de dados do domínio

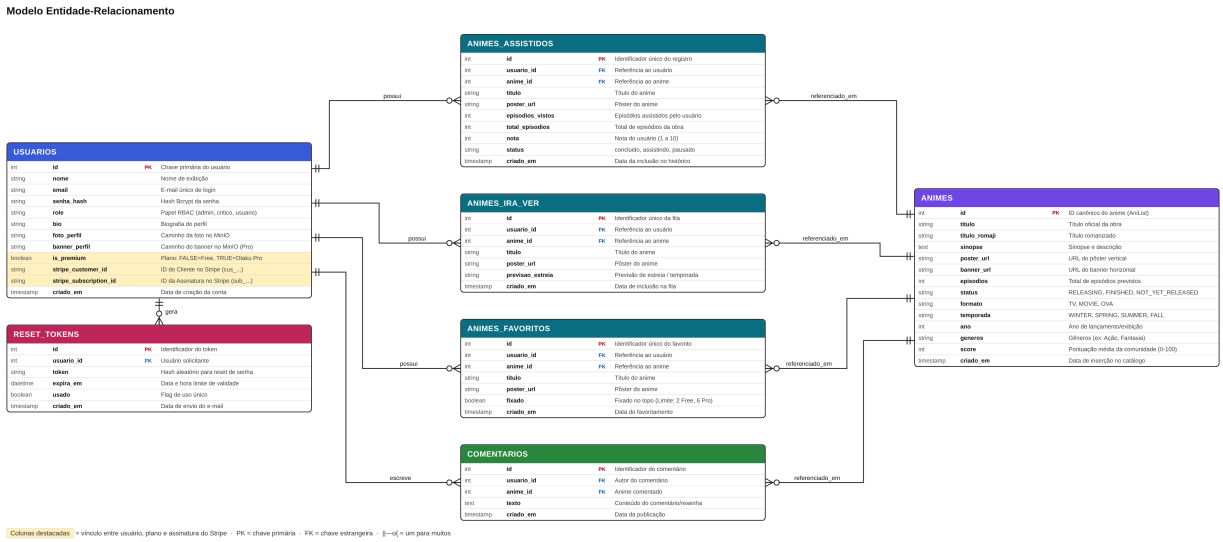


Figura 4: Modelo de dados do domínio

3.6 Segurança

Ação	admin	critico	usuario
cadastrar e editar animes no acervo	sim	sim	não
ver os dados e logs de outro usuário	sim	não	não
excluir animes do catálogo	sim	não	não
moderar/excluir comentários de terceiros	sim	não	não
publicar resenhas críticas em destaque	sim	sim	não
gerenciar próprias listas (assistidos, fila, favoritos)	sim	sim	sim
publicar e excluir próprios comentários	sim	sim	sim
editar perfil e enviar fotos/banners	sim	sim	sim

3.6.1 Isolamento de Dados e Gestão de Segredos

Isolamento por Usuário nas Consultas (Prevenção de IDOR):

- **Identidade via JWT:** O `id` do usuário nunca vem do corpo da requisição (`req.body`); ele é extraído com segurança da assinatura do token JWT decodificado no middleware (`req.user.id`).
- **Filtro Forçado no SQL:** Todas as consultas de listas pessoais (`animes_assistidos`, `animes_ira_ver`, `animes_favoritos`) amarram obrigatoriamente a cláusula `WHERE usuario_id = ?` usando *prepared statements*. Isso impede que um usuário leia, edite ou exclua registros alheios.
- **Validação de Autoria:** Em dados públicos (comentários), a exclusão valida se `comment.usuario_id === req.user.id`. Apenas o papel `admin` possui a permissão especial de moderar comentários de terceiros (`comentarios:excluir_qualquer`).

Gestão e Proteção de Segredos:

- **Variáveis de Ambiente na Stack:** Credenciais do MariaDB, `JWT_SECRET`, chaves do Stripe e tokens SMTP residem apenas na memória em tempo de execução, injetadas pelo Docker Compose / Portainer.
- **Blindagem do Git:** O arquivo real `.env` está explicitamente ignorado no `.gitignore` e no `.dockerignore`, impedindo qualquer vazamento no GitHub ou dentro das imagens Docker.
- **Gabarito `.env.example`:** Apenas o modelo `.env.example` é versionado no repositório, contendo exclusivamente os nomes das chaves com

valores fictícios de exemplo.
- **Rede Interna:** MariaDB, Redis e MinIO não expõem portas para a internet (`expose` em vez de `ports`), comunicando-se apenas na rede interna do Docker (`internal-net`).

3.7 Observabilidade, CI/CD e documentação da API

Não realizado.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no seu projeto	Quem gerencia
IaaS	A VM Linux (GCP), capacidade computacional (vCPU/RAM), disco em bloco, rede/firewall e subdomínio DNS (`felipe-shida-isw055.lapps.studio`) onde toda a stack é executada.	Professor / Fatec (em conjunto com o fornecedor GCP)
PaaS	Ambiente de execução de containers (Docker + Portainer) e serviços de dados em container: MariaDB (banco relacional), Redis (streams de logs) e MinIO (object storage S3).	Compartilhado: Professor / Fatec (Docker/Portainer e MariaDB) e você (stacks, volumes, Redis e MinIO)
SaaS	O software final **AniTrack** entregue pronto via navegador para o usuário otaku; e os SaaS terceiros integrados: Stripe (pagamentos), GitHub (repositório) e Mailtrap/Brevo (e-mails transacionais).	Você (gerencia o produto AniTrack) e Fornecedores externos (gerenciam Stripe, GitHub e Brevo)

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	21/10/2026 18:36	entregue com atraso
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	24/10/2026 13:27	entregue com atraso
9	Fork e plano de negócio	23/10/2026	05/11/2026 16:49	entregue com atraso

Nº	Atividade	Data planejada	Data realizada	Situação
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	07/11/2026 10:51	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	10/11/2026 21:21	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	21/10/2026 18:36
Situação	entregue com atraso
Evidência	GitHub — commit + print do checkout concluído e do usuário premium
Link	https://github.com/FeYuzo/catalogo-de-filmes-api/commit/ffc16525091bed54684d0221fcae43811c903626

Nesta atividade, implementei a oferta do Plano Premium no catálogo de filmes, com checkout de assinatura pelo Stripe e atualização do status da conta por webhook. A integração foi configurada por variáveis de ambiente, e o backend passou a disponibilizar rotas autenticadas para iniciar o checkout e consultar a assinatura.

Também defini a regra de negócio dos planos: contas gratuitas podem salvar até cinco filmes favoritos, enquanto assinantes Premium têm favoritos ilimitados. Ajustei a interface para apresentar esse limite e o fluxo de assinatura, e registrei eventos relevantes na auditoria.

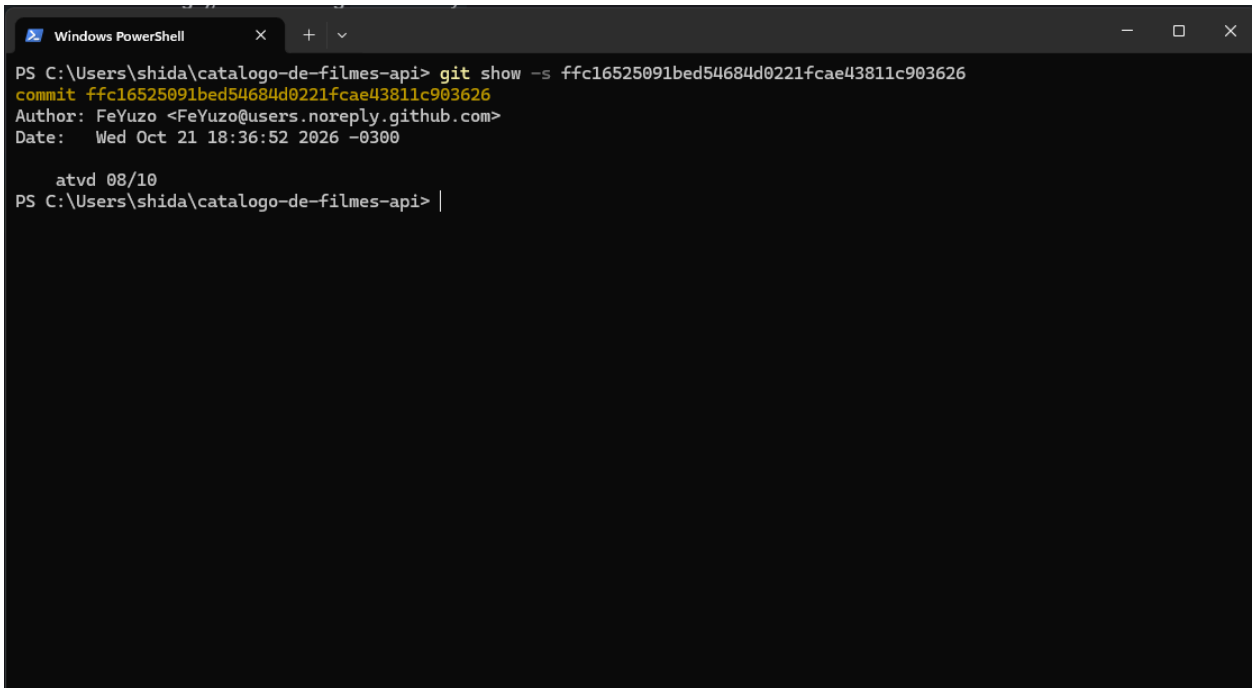


Figura 5: Atividade 7 — evidência da entrega (link, data e hora visíveis)

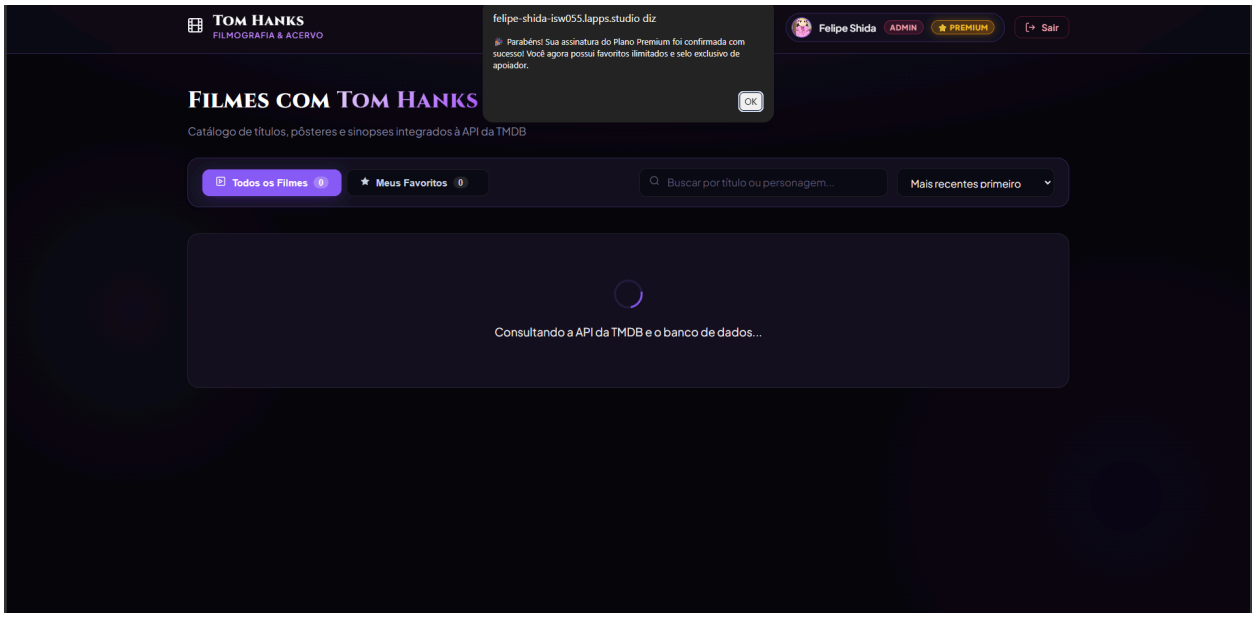


Figura 6: Atividade 7 — print do checkout concluído e do usuário premium

A principal dificuldade foi integrar o checkout do Stripe com o status Premium armazenado no banco e garantir que os limites de favoritos refletissem o plano de cada usuário. Resolvi isso configurando a integração por variáveis de ambiente, validando os eventos recebidos por webhook e usando o status da conta para aplicar os limites correspondentes na API e na interface.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026

Data realizada	24/10/2026 13:27
Situação	entregue com atraso
Evidência	GitHub — commit + DOCUMENTACAO.md linkado no README
Link	https://github.com/FeYuzo/catalogo-de-filmes-api/commit/a1d78cfb7808f7ebed05eb2874d9e07c20d9e554

Nesta atividade, ampliei o README.md com uma documentação reversa do sistema: classificação dos serviços usados em SaaS, PaaS e IaaS, visão funcional da aplicação e descrição da arquitetura de microsserviços e suas conexões.

Também documentei as principais regras de negócio. Como autenticação e recuperação de senha, limite de favoritos no plano gratuito, recursos premium, moderação, perfil, pagamentos e auditoria, além do modelo de dados. A decisão foi reunir essas informações na documentação existente, explicando o comportamento e as tecnologias a partir da estrutura do projeto, sem alterar o código da aplicação.

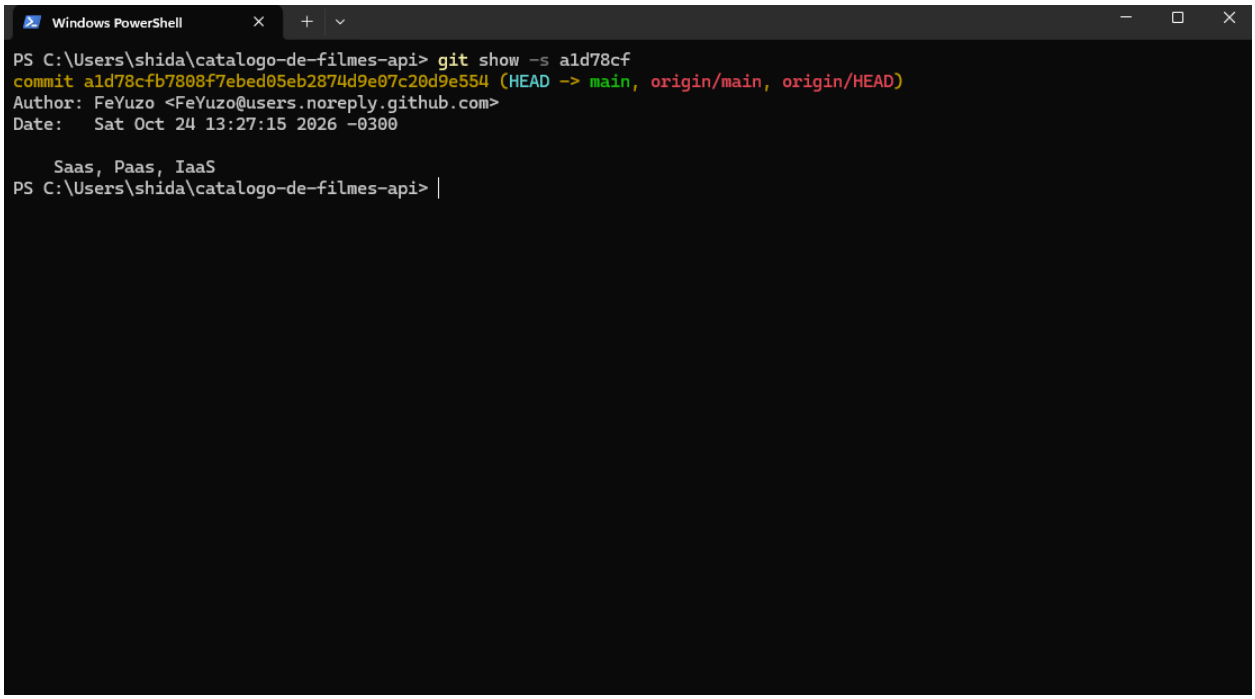
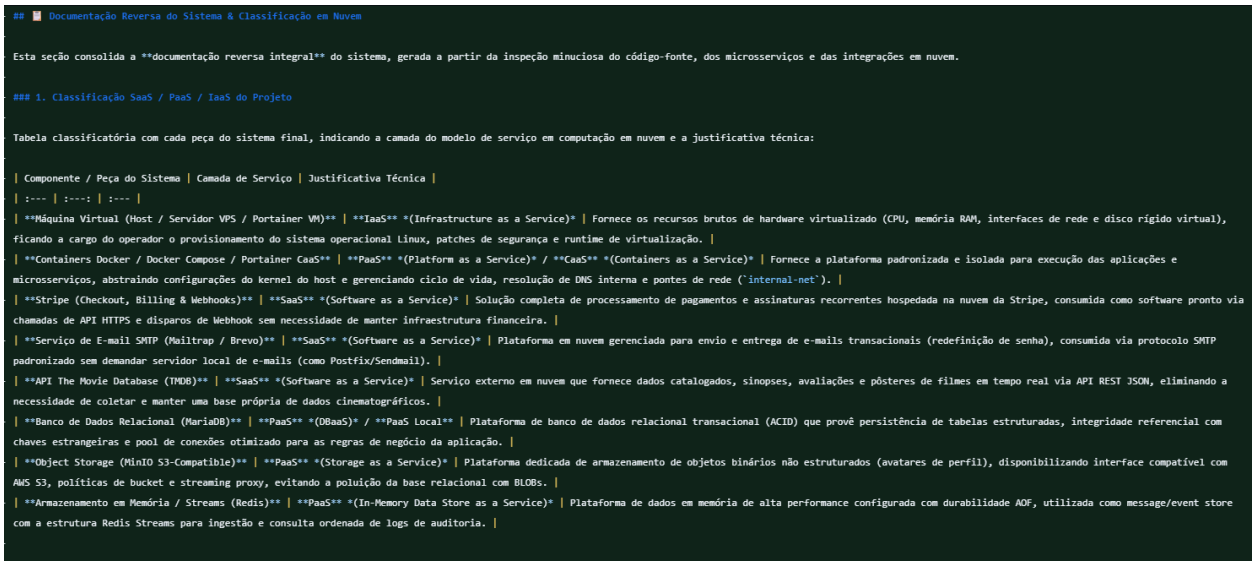


Figura 7: Atividade 8 — evidência da entrega (link, data e hora visíveis)



📋 Documentação Reversa do Sistema & Classificação em Nuvem

Esta seção consolida a **documentação reversa integral** do sistema, gerada a partir da inspeção minuciosa do código-fonte, dos microsserviços e das integrações em nuvem.

1. Classificação SaaS / PaaS / IaaS do Projeto

Tabela classificatória com cada peça do sistema final, indicando a camada do modelo de serviço em computação em nuvem e a justificativa técnica:

Componente / Peça do Sistema	Camada de Serviço	Justificativa Técnica
Máquina Virtual (Host / Servidor VPS / Portainer VM)	**IaaS** *(Infrastructure as a Service)*	Fornece os recursos brutos de hardware virtualizado (CPU, memória RAM, interfaces de rede e disco rígido virtual), ficando a cargo do operador o provisionamento do sistema operacional Linux, patches de segurança e runtime de virtualização.
Containers Docker / Docker Compose / Portainer CaaS	**PaaS** *(Platform as a Service)* / **CaaS** *(Containers as a Service)*	Fornece a plataforma padronizada e isolada para execução das aplicações e microsserviços, abstraindo configurações do kernel do host e gerenciando ciclo de vida, resolução de DNS interna e pontes de rede (`internal-net`).
Stripe (Checkout, Billing & Webhooks)	**SaaS** *(Software as a Service)*	Solução completa de processamento de pagamentos e assinaturas recorrentes hospedada na nuvem da Stripe, consumida como software pronto via chamadas de API HTTPS e disparos de Webhook sem necessidade de manter infraestrutura financeira.
Serviço de E-mail SMTP (Mailtrap / Brevo)	**SaaS** *(Software as a Service)*	Plataforma em nuvem gerenciada para envio e entrega de e-mails transacionais (redefinição de senha), consumida via protocolo SMTP padronizado sem demandar servidor local de e-mails (como Postfix/Sendmail).
API The Movie Database (TMDB)	**SaaS** *(Software as a Service)*	Serviço externo em nuvem que fornece dados catalogados, sinopses, avaliações e pôsteres de filmes em tempo real via API REST JSON, eliminando a necessidade de coletar e manter uma base própria de dados cinematográficos.
Banco de Dados Relacional (MariaDB)	**PaaS** *(DBaaS)* / **PaaS Local**	Plataforma de banco de dados relacional transacional (ACID) que provê persistência de tabelas estruturadas, integridade referencial com chaves estrangeiras e pool de conexões otimizado para as regras de negócio da aplicação.
Object Storage (MinIO S3-Compatible)	**PaaS** *(Storage as a Service)*	Plataforma dedicada de armazenamento de objetos binários não estruturados (avatares de perfil), disponibilizando interface compatível com AWS S3, políticas de bucket e streaming proxy, evitando a poluição da base relacional com BLOBs.
Armazenamento em Memória / Streams (Redis)	**PaaS** *(In-Memory Data Store as a Service)*	Plataforma de dados em memória de alta performance configurada com durabilidade AOF, utilizada como message/event store com a estrutura Redis Streams para ingestão e consulta ordenada de logs de auditoria.

Figura 8: Atividade 8 — README

A principal dificuldade foi reunir informações espalhadas entre os microsserviços e entender como as regras de negócio e os serviços em nuvem se conectam. Resolvi isso analisando o código e as configurações do projeto e organizando as descobertas por tema no README.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	05/11/2026 16:49
Situação	entregue com atraso
Evidência	GitHub — issue de entrega (@siriani) + repositório novo + Nome Frase, Dor e Persona + print do histórico de commits e das issues
Link	https://github.com/FeYuzo/Animes-API/commit/8ba416293d4659ed059c052cda2fdfb162d5c70d

Nesta atividade, estruturei o plano de negócio do AniTrack, posicionando-o como uma plataforma SaaS de gestão de animes e simulcast. Defini o problema e o público-alvo, a proposta de valor, as alternativas concorrentes, o modelo de receita com planos e Stripe, as métricas de sucesso e o Lean Canvas. A decisão central foi evoluir o catálogo original para um serviço com benefícios premium e potencial de monetização.

Também documentei a arquitetura de microsserviços, o isolamento de rede, o modelo relacional do banco de dados, as requisições da API e as instruções para executar o projeto localmente. Essas definições conectam a proposta de negócio à estrutura técnica prevista para a plataforma.

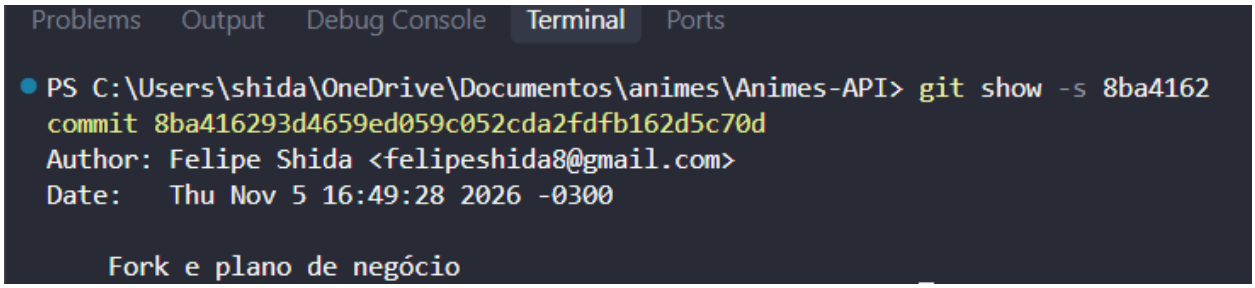


Figura 9: Atividade 9 — evidência da entrega (link, data e hora visíveis)

1. Nome e Frase

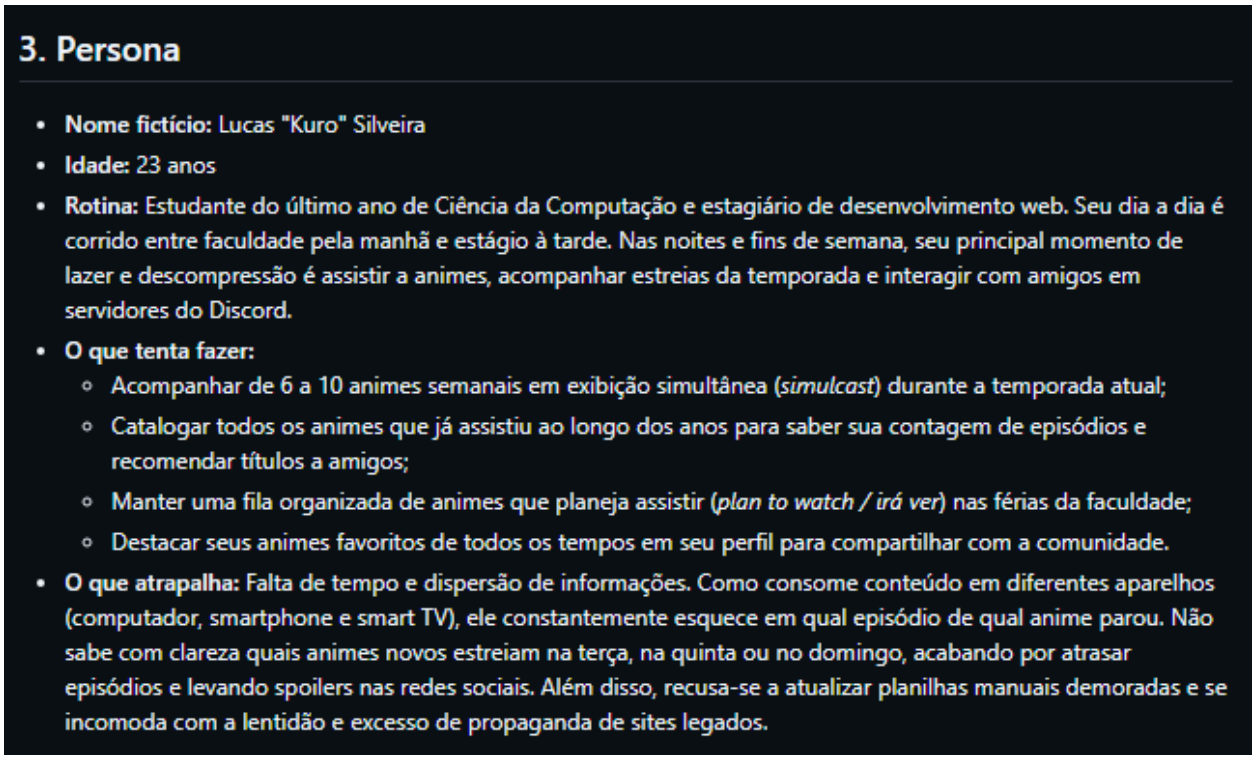
- **Nome do Produto: AniTrack** (ou **AniTrack API & SaaS**)
- **Frase de Uma Linha:** *O AniTrack é uma plataforma e API SaaS completa para gerenciar animes assistidos, planejar animes que irá ver, favoritar títulos marcantes e acompanhar o calendário de lançamentos simulcast em tempo real para fãs e comunidades otaku.*

Figura 10: Atividade 9 — nome e frase.

2. A Dor

- **Quem sofre:** Fãs, espectadores assíduos e membros de comunidades de animes (de 16 a 35 anos) que acompanham simultaneamente múltiplos animes sazonais a cada nova temporada (*seasonal simulcasts*), além de consumidores casuais que querem manter um registro confiável do que assistem.
- **Quando:** Semanalmente nos dias de transmissão simultânea (simulcast) e nos períodos de virada de temporada (inverno, primavera, verão e outono), ao tentar lembrar em qual episódio parou, quais obras estreiam em cada dia da semana e quais títulos ainda estão na fila de espera.
- **Quanto custa hoje:**
  - **Dinheiro:** Gasto acumulado com múltiplos serviços de streaming sem saber com antecedência em qual plataforma ou dia da semana o anime é exibido, além de assinaturas de aplicativos de terceiros repletos de anúncios invasivos.
  - **Tempo:** Perda estimada de 30 a 50 minutos semanais navegando entre múltiplos sites, redes sociais, notas de celular e fóruns em busca de datas de transmissão, numeração de episódios e listas de obras.
  - **Risco:** Alto risco de receber *spoilers* em redes sociais (TikTok, X/Twitter, Instagram) por perder o dia da transmissão do episódio; abandono involuntário de bons animes por esquecer em qual temporada ou episódio parou; e perda irremediável de histórico de anos ao trocar de smartphone ou apagar anotações não centralizadas.
- **Como essa pessoa resolve hoje:** Faz anotações desestruturadas no bloco de notas do celular, manda mensagens para si mesma em conversas do WhatsApp, mantém planilhas manuais no Excel/Google Sheets (que exigem preenchimento manual cansativo e não possuem integração de dados) ou recorre a plataformas legadas (como MyAnimeList), que possuem interfaces pesadas, defasadas, cheias de banners publicitários e com experiência mobile ruim.

Figura 11: Atividade 9 — dor.



3. Persona

- **Nome fictício:** Lucas "Kuro" Silveira
- **Idade:** 23 anos
- **Rotina:** Estudante do último ano de Ciência da Computação e estagiário de desenvolvimento web. Seu dia a dia é corrido entre faculdade pela manhã e estágio à tarde. Nas noites e fins de semana, seu principal momento de lazer e descompressão é assistir a animes, acompanhar estreias da temporada e interagir com amigos em servidores do Discord.
- **O que tenta fazer:**
  - Acompanhar de 6 a 10 animes semanais em exibição simultânea (*simulcast*) durante a temporada atual;
  - Catalogar todos os animes que já assistiu ao longo dos anos para saber sua contagem de episódios e recomendar títulos a amigos;
  - Manter uma fila organizada de animes que planeja assistir (*plan to watch / irá ver*) nas férias da faculdade;
  - Destacar seus animes favoritos de todos os tempos em seu perfil para compartilhar com a comunidade.
- **O que atrapalha:** Falta de tempo e dispersão de informações. Como consome conteúdo em diferentes aparelhos (computador, smartphone e smart TV), ele constantemente esquece em qual episódio de qual anime parou. Não sabe com clareza quais animes novos estreiam na terça, na quinta ou no domingo, acabando por atrasar episódios e levando spoilers nas redes sociais. Além disso, recusa-se a atualizar planilhas manuais demoradas e se incomoda com a lentidão e excesso de propaganda de sites legados.

Figura 12: Atividade 9 — persona.

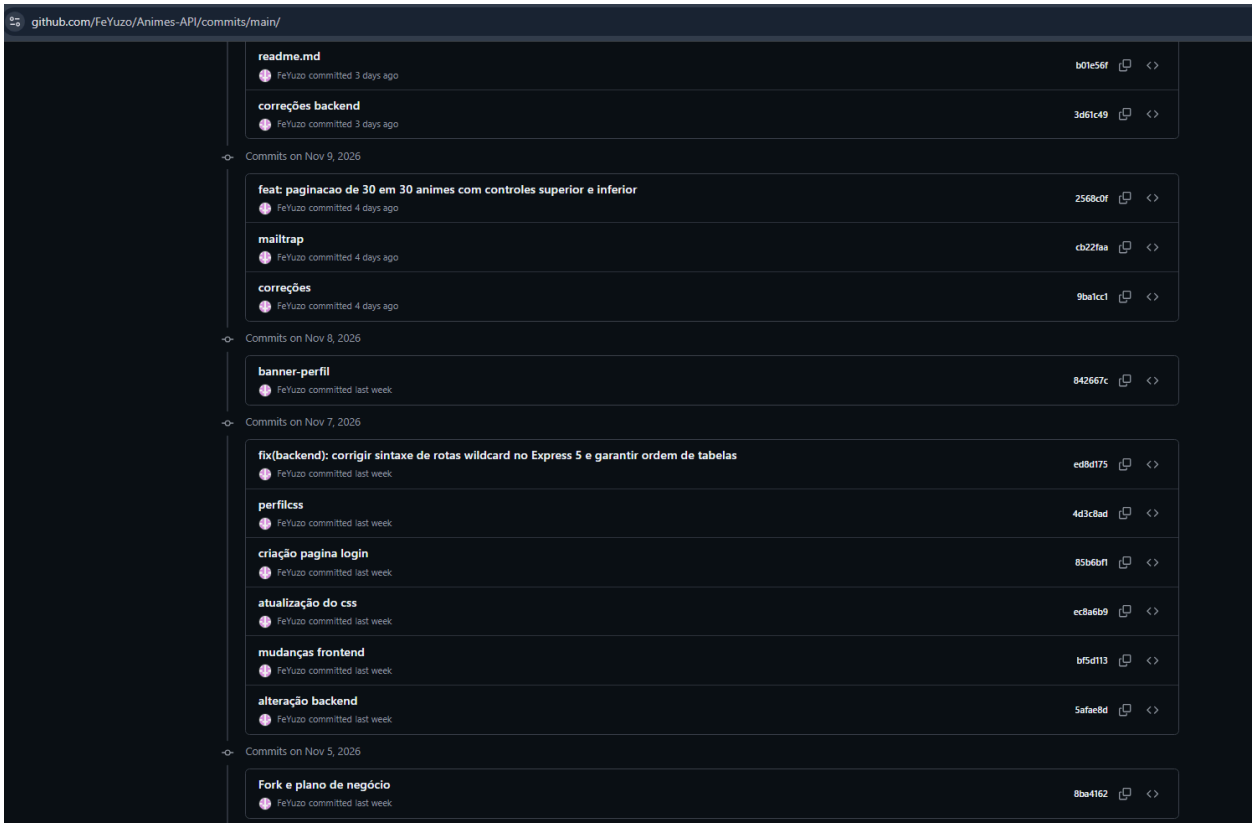


Figura 13: Atividade 9 — histórico de commits

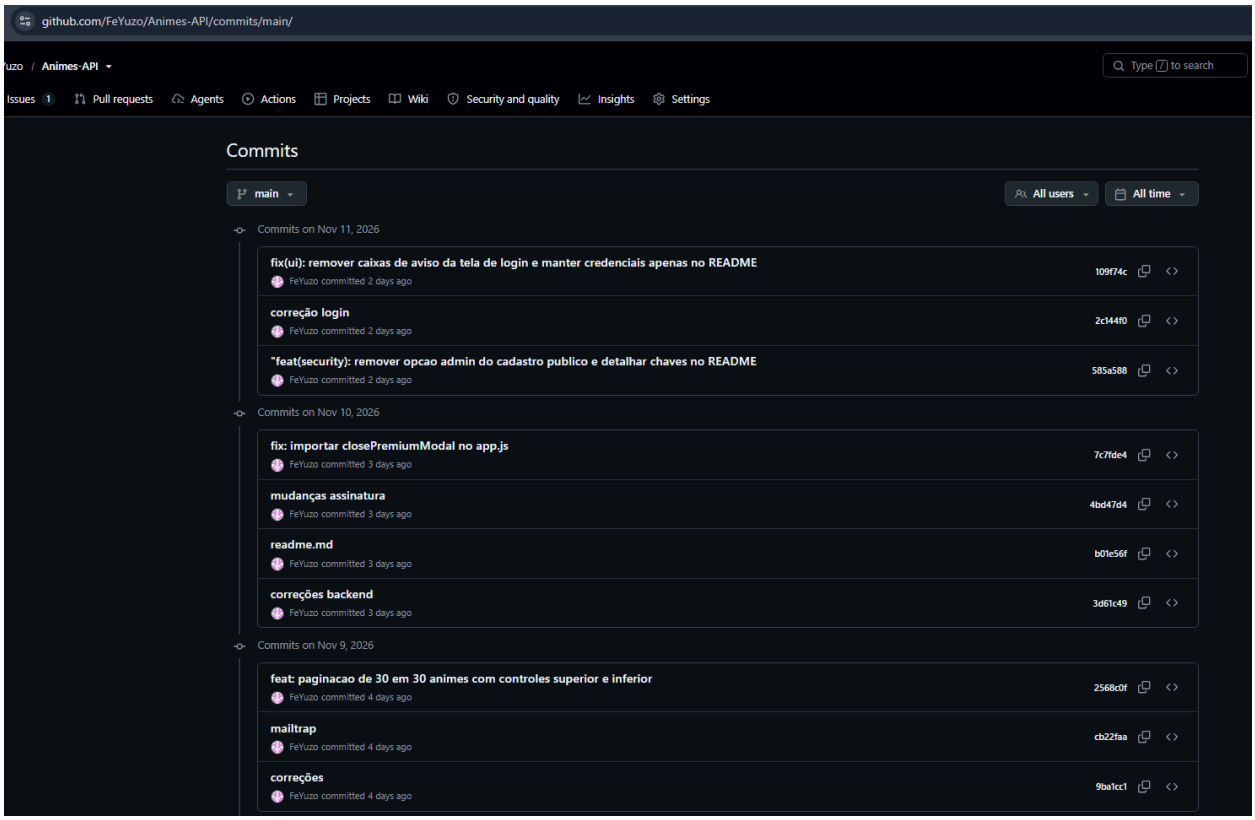


Figura 14: Atividade 9 — histórico de commits

A principal dificuldade foi conectar o plano de negócio à arquitetura técnica sem perder a coerência entre os recursos oferecidos e a proposta de valor. Resolvi isso organizando a documentação por etapas — público e modelo de receita, seguidos de API, microsserviços e dados — para deixar claro como a solução técnica sustenta o AniTrack.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	07/11/2026 10:51
Situação	entregue com atraso
Evidência	GitHub — issue de entrega (@siriani) + commit + URL no ar + prints da landing, 403 e dos eventos no log
Link	https://github.com/FeYuzo/Animes-API/commit/5afae8d382a466d0e85ef1a9ed1ed0d369a3e326

Nesta atividade, ampliei o backend do AniTrack para integrar o catálogo e os dados de simulcast da AniList, incluindo busca e detalhes de animes. Também implementei endpoints para gerenciar animes assistidos, a lista “quero assistir” e favoritos, além de comentários por anime.

A decisão foi concentrar esses recursos em rotas próprias para animes e usar autenticação nas operações pessoais, mantendo algumas consultas públicas. Também ajustei a conexão com o banco, a configuração por variáveis de ambiente e o Docker para apoiar a execução do serviço.

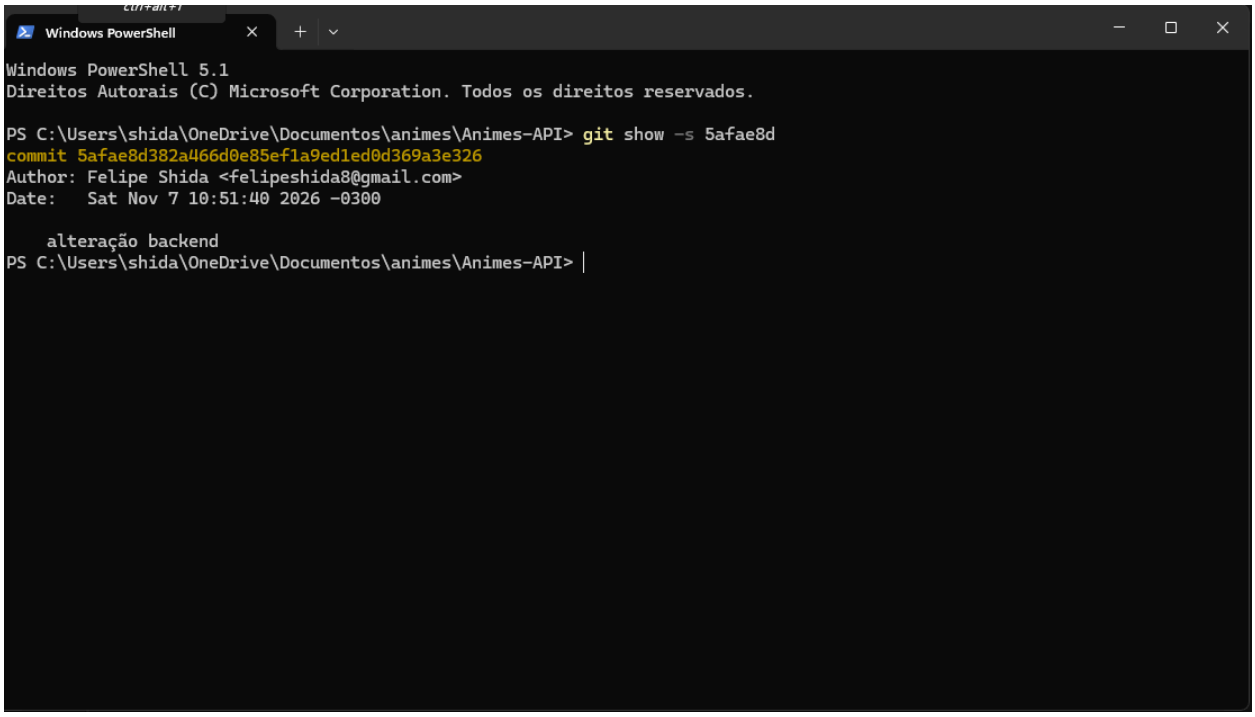


Figura 15: Atividade 10 — evidência da entrega (link, data e hora visíveis)

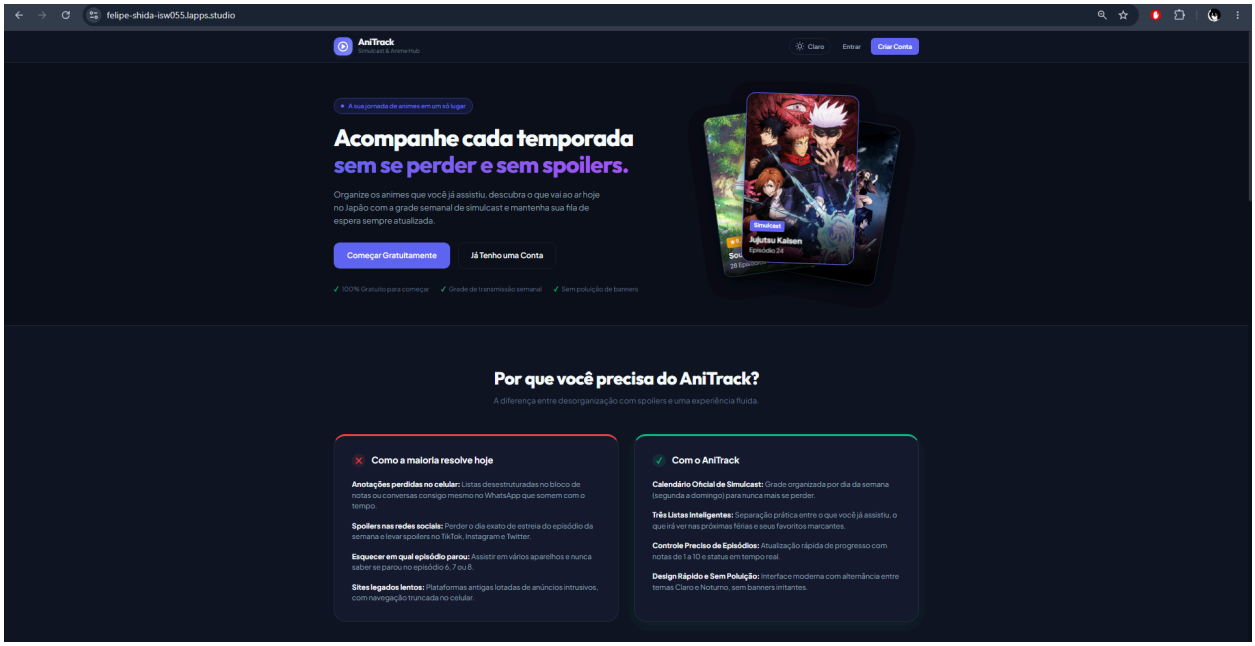


Figura 16: Atividade 10 — prints da landing

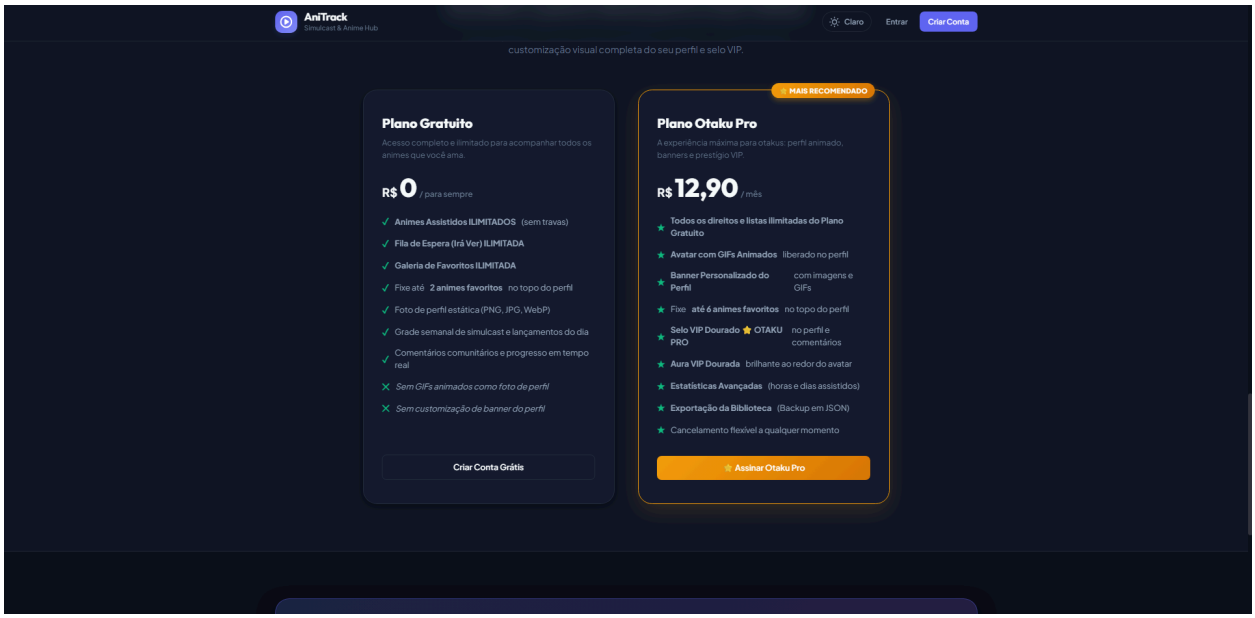


Figura 17: Atividade 10 — prints da landing

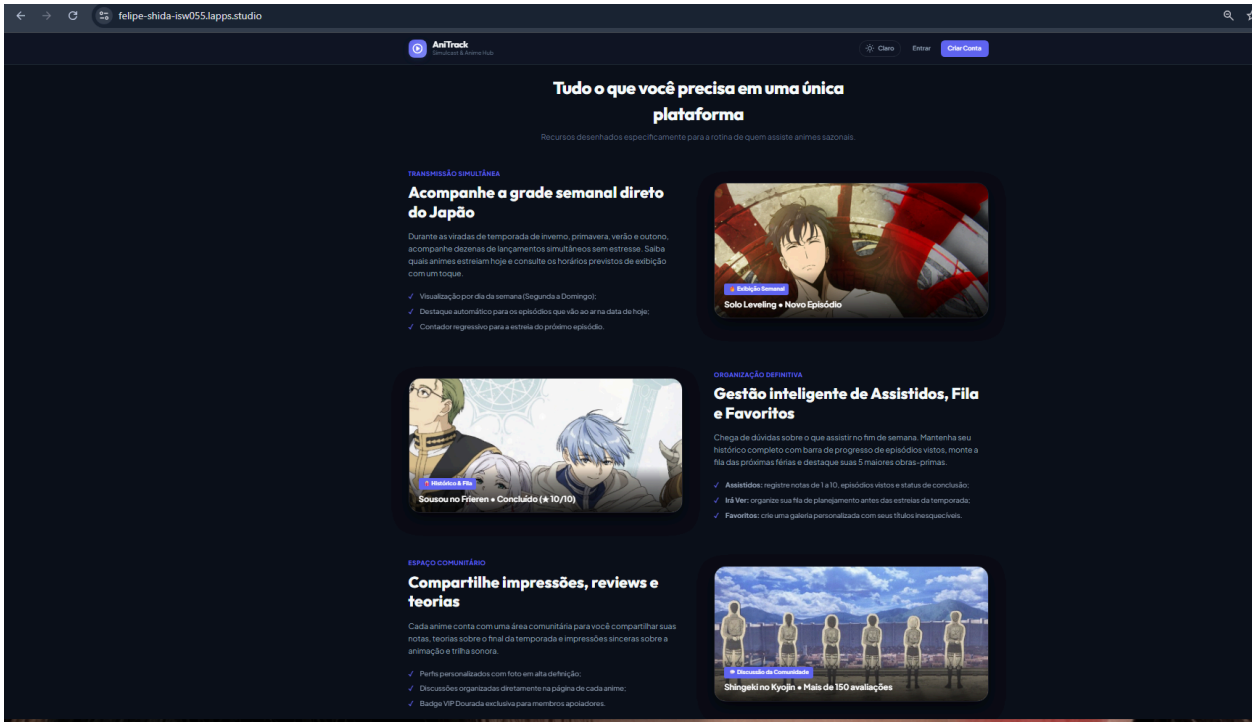


Figura 18: Atividade 10 — prints da landing

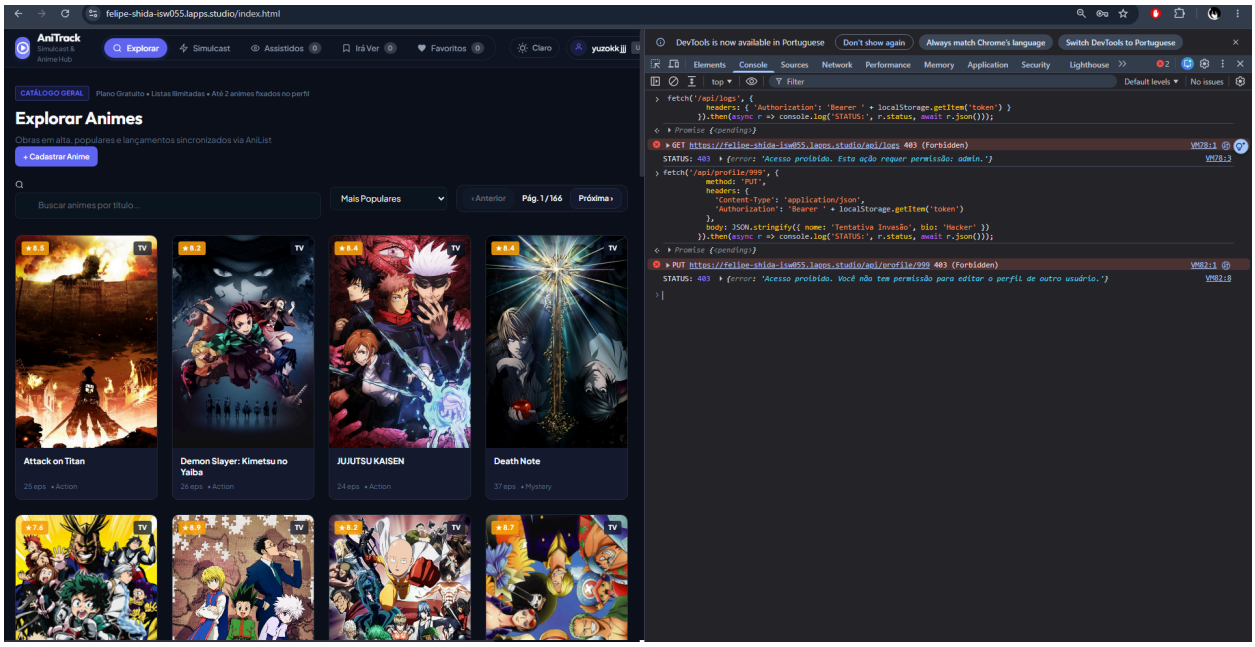


Figura 19: Atividade 10 — 403

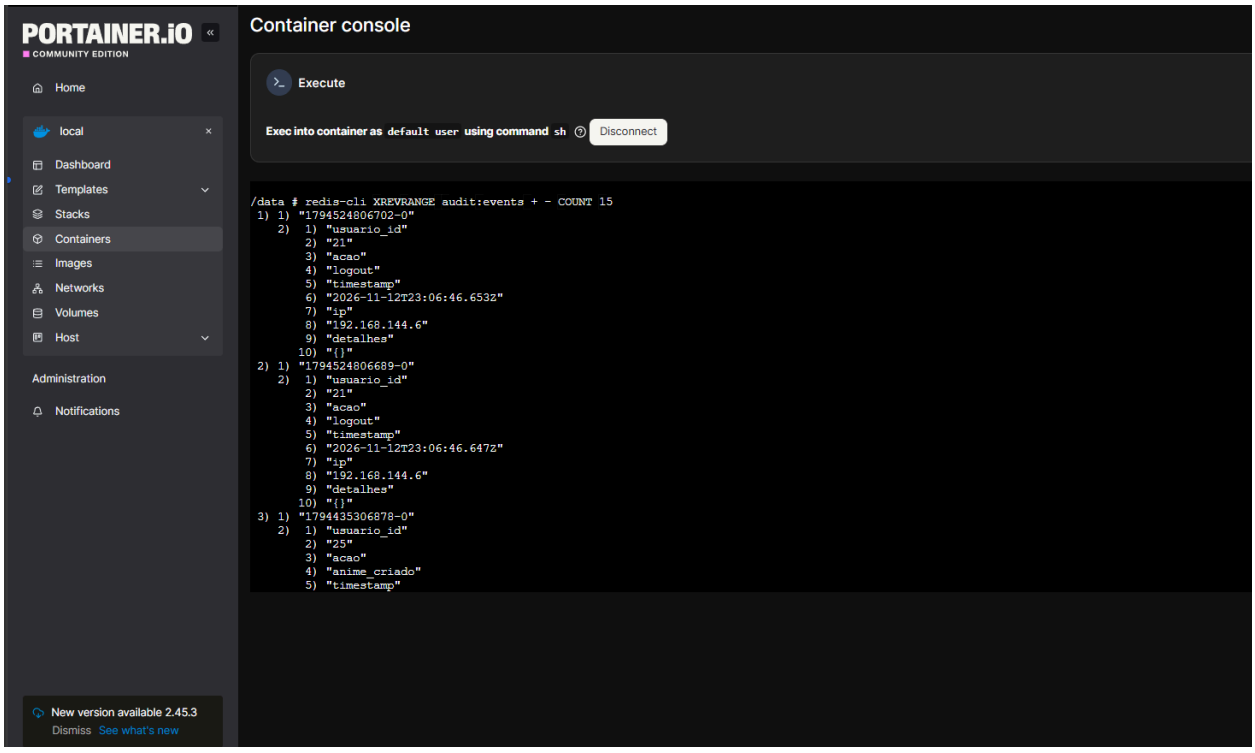


Figura 20: Atividade 10 — logs

A principal dificuldade foi adaptar o backend, originalmente voltado a filmes, para trabalhar com catálogo e simulcast de animes sem perder recursos como favoritos, listas pessoais e comentários. Resolvi essa transição integrando a AniList e organizando rotas e controladores específicos para animes, além de ajustar a configuração do banco e do ambiente de execução.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano e portal do cliente.
Data planejada	13/11/2026
Data realizada	10/11/2026 21:21
Situação	entregue
Evidência	GitHub — issue de entrega (@siriani) + commit + prints da assinatura com o cartão 4242, do Stripe em modo de teste e do webhook.
Link	https://github.com/FeYuzo/Animes-API/commit/4bd47d438091f1e894d3f4355448153003672acc

Nesta atividade, aprimorei a integração da assinatura Premium com o Stripe: o backend passou a oferecer rotas autenticadas para criar o checkout, verificar o resultado da compra, consultar o status da assinatura e solicitar seu cancelamento, além de processar notificações do Stripe por webhook.

Também conectei esses recursos à interface do AniTrack, atualizando a comunicação com a API e os elementos relacionados ao plano Premium. A decisão foi permitir que o usuário acompanhe e gerencie a assinatura dentro da própria plataforma, enquanto o webhook mantém o estado da assinatura sincronizado com o Stripe.

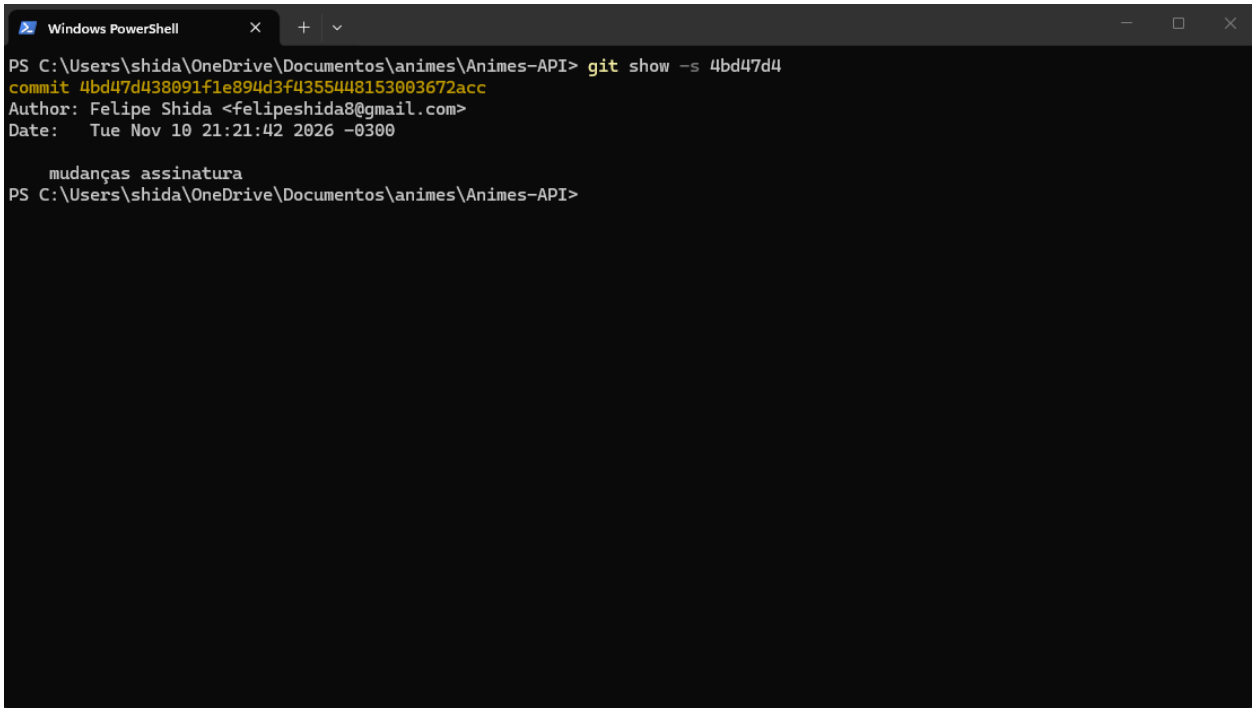


Figura 21: Atividade 11 — evidência da entrega (link, data e hora visíveis)

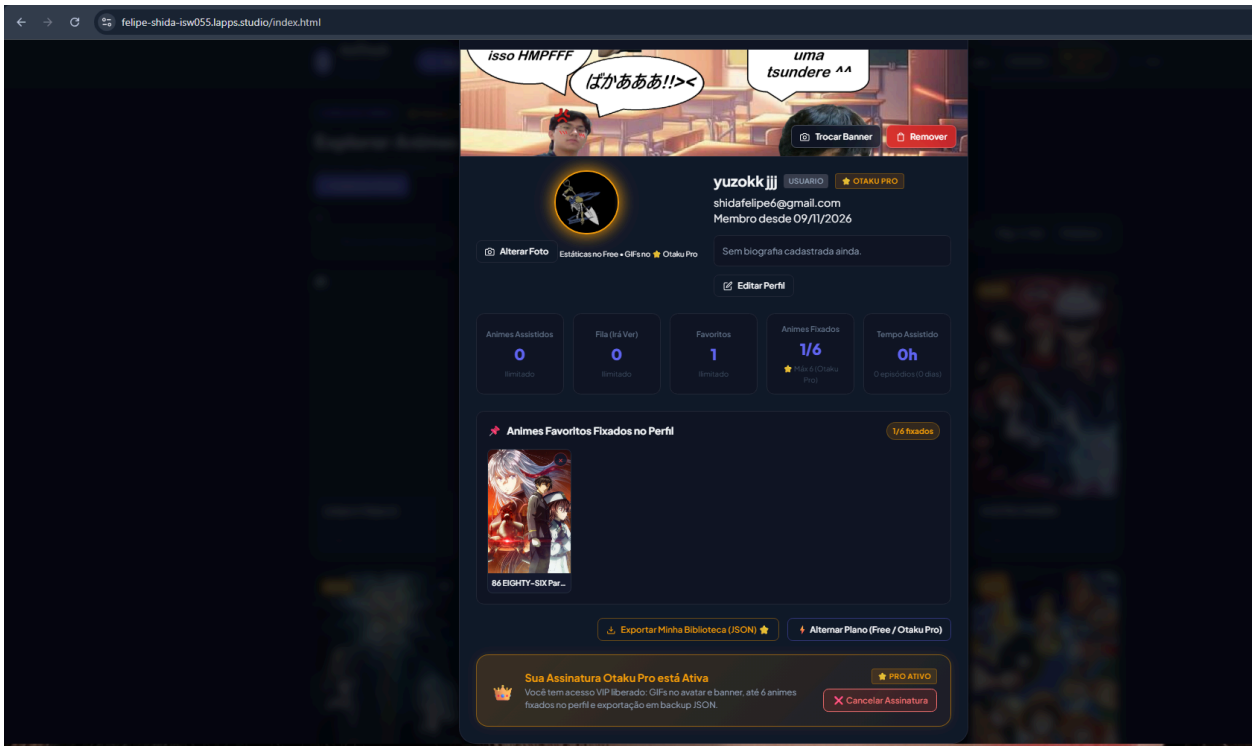


Figura 22: Atividade 11 — botão cancelar assinatura

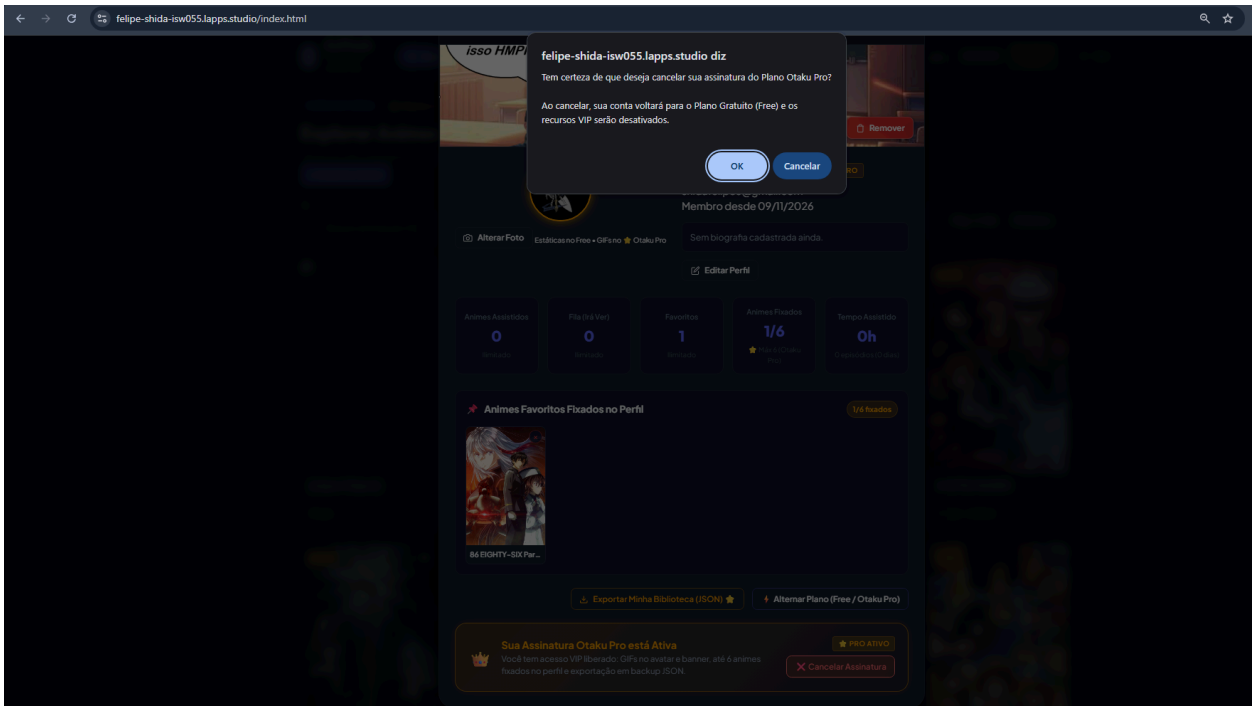


Figura 23: Atividade 11 —cancelando assinatura

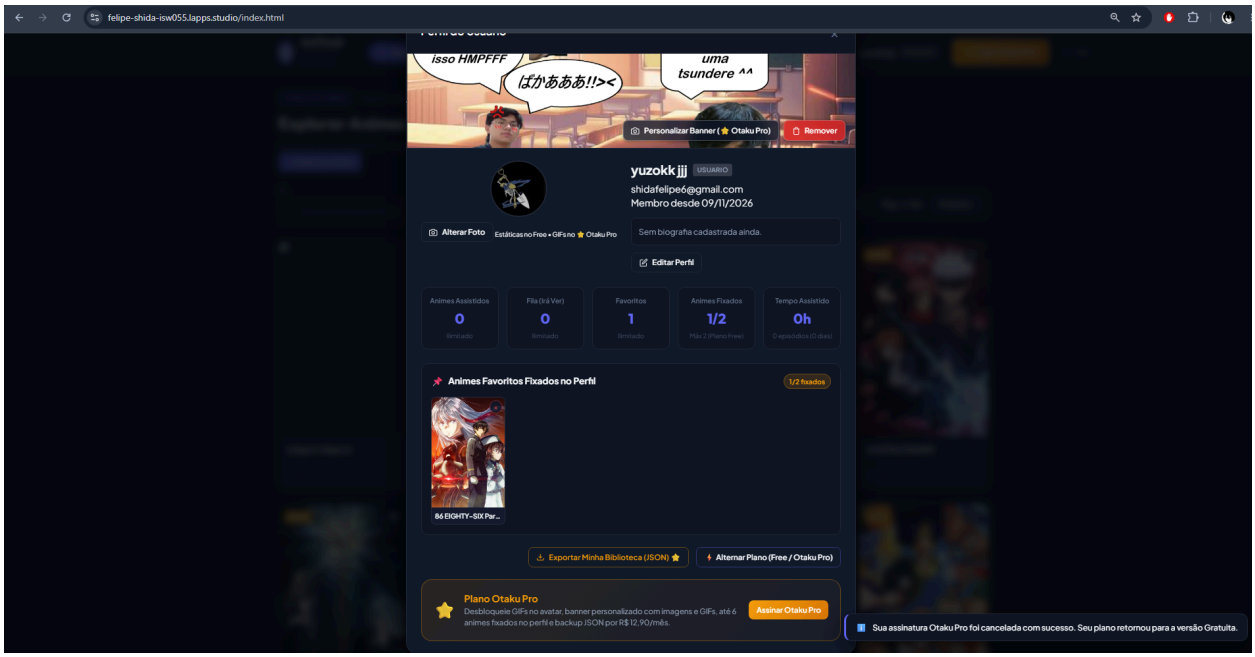


Figura 24: Atividade 11 —assinatura cancelada.

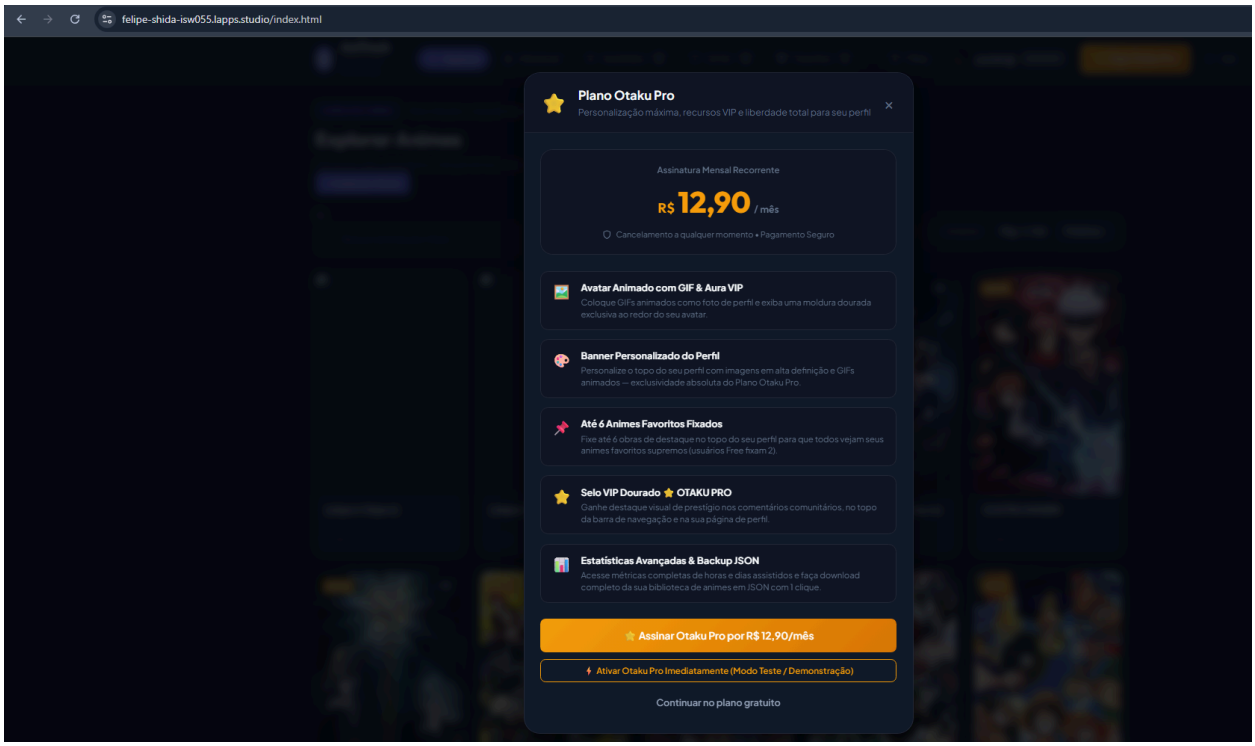


Figura 25: Atividade 11 —assinatura nova.

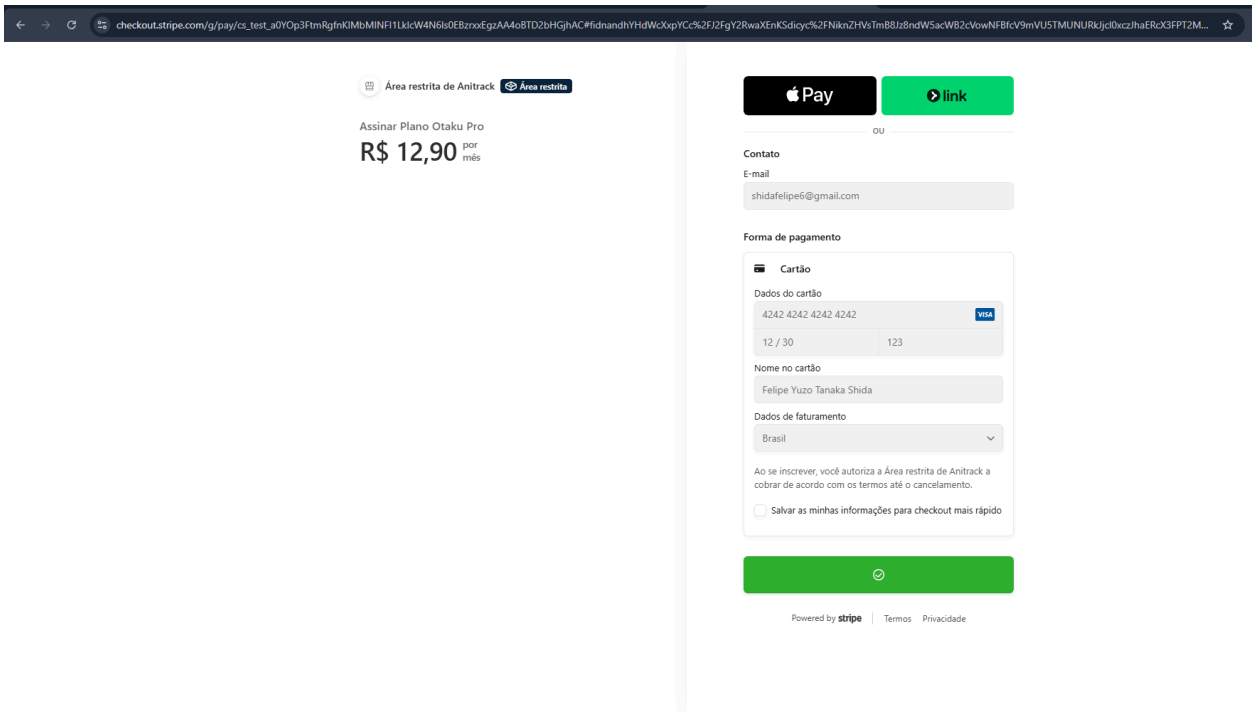


Figura 26: Atividade 11 —assinatura com cartão 4242.

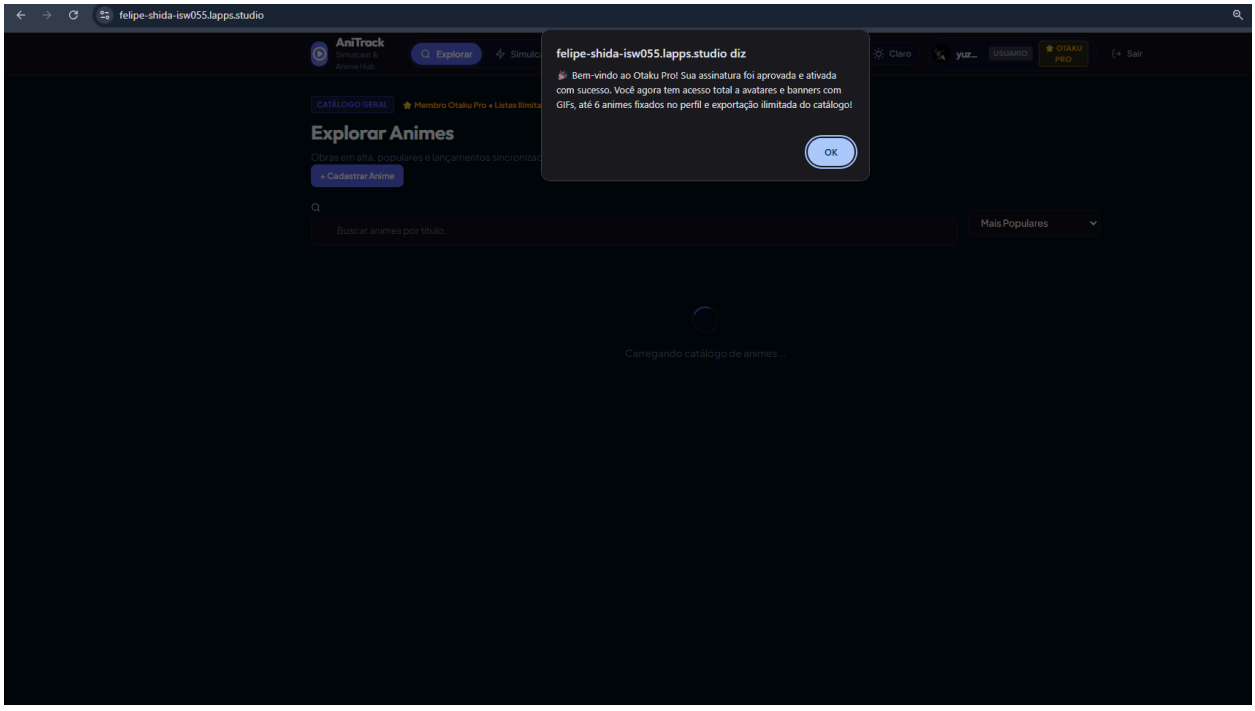


Figura 27: Atividade 11 —webhook mostrando sucesso na assinatura.

A principal dificuldade foi manter o status Premium consistente entre o Stripe, o backend e a interface, inclusive após o retorno do checkout. Para resolver, foram adicionados endpoints de verificação e consulta de assinatura, junto ao processamento de webhooks, e a interface foi conectada a essas informações para refletir o estado atualizado.

6 Considerações finais

Ao longo do desenvolvimento do AniTrack neste bimestre, o aprendizado mais transformador foi a passagem prática de uma mentalidade monolítica para o design de uma solução SaaS distribuída e conteinerizada. Consolidar na prática o isolamento perimetral em redes Docker, a ingestão de eventos comportamentais em alta performance via Redis Streams e o armazenamento de objetos compatível com S3 (MinIO) deu clareza sobre como projetar sistemas escaláveis e desacoplados. O maior desafio técnico esteve na confiabilidade das integrações e na segurança inter-serviços: orquestrar a comunicação interna sem expor portas desnecessárias ao host, validar a assinatura criptográfica dos webhooks do Stripe preservando o payload bruto (`rawBody`) e garantir a idempotência nas mutações de estado no MariaDB exigiram atenção rigorosa à consistência e à tolerância a falhas.

Analisando criticamente as decisões de arquitetura, faria diferente a estruturação do ponto de entrada: em vez de concentrar o roteamento e o proxy de arquivos estáticos no próprio container do catálogo, adotaria um API Gateway dedicado (como Nginx ou Traefik) na borda da stack para gerenciar terminação TLS, rate limiting e balanceamento de carga de forma isolada. Além disso, estruturaria consumidores dedicados (*workers* assíncronos) para descarregar o processamento de webhooks de pagamento e o disparo transacional de e-mails em segundo plano, evitando que latências pontuais de serviços SaaS externos afetassem o ciclo de resposta HTTP da API.

Para que o AniTrack atinja maturidade de um produto comercial pronto para o mercado, ainda faltam etapas cruciais de governança e infraestrutura: a criação de uma esteira automatizada de CI/CD com testes de carga e regressão ponta a ponta; a implementação de certificados SSL/TLS automáticos (HTTPS) no domínio; rotinas automatizadas de backup e replicação para o MariaDB e buckets do MinIO; e recursos nativos de engajamento, como notificações push em tempo real (web/mobile) alertando os assinantes no instante exato em que os episódios de simulcast vão ao ar.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Felipe Yuzo Tanaka Shida

Pompeia, 13/11/2026